KARKAII SCHOOL ERP

The 20-hour backend build: Sign up, Sign in, Schools & Permissions

A step-by-step plan to build the Node.js backend that your React frontend can use for sign up, sign in, school creation, roles and user permissions — on your DigitalOcean PostgreSQL database. You write the code; every step tells you what to build, how to check it, and what to do if it fails.

Updated 29 October 2026 · Node 22 · Express 5 · TypeScript · Prisma 6 · PostgreSQL (DigitalOcean)
Written against your real frontend (`karkaii-frontend`): the response shapes, cookie and role names below come from its code.

0. Read this first

The goal (and what “done” means)

When you finish, you can open your React app, **sign up**, **create a school**, **sign in** as people with different roles, and the UI shows or hides features according to **permissions** — with real data coming from your backend and DigitalOcean database. Dashboards and other modules stay on mock data for now.

The 20-hour plan

Tags: MUST = needed for the demo · SHOULD = do if on schedule · COULD = only with spare time. If you are behind, drop COULD items first, then SHOULD.

Hours	Phase	What you finish	Tag
0:00–0:45	**1.** Database & project	DigitalOcean database ready and reachable; project created (chapters C0, C1)	MUST
0:45–3:15	**2.** Fast-track learning	Part A: read the starred (★) sections, do exercises 1, 4, 7 (2.5 h)	MUST
3:15–5:45	**3.** Data model & foundations	9 tables in PostgreSQL, config, error handling, skeleton app (C2–C4)	MUST
5:45–8:45	**4.** Sessions, sign up, sign in	Tokens + refresh cookie, `/signup`, `/login`, `/me`, `/refresh`, `/logout` (C5–C9)	MUST
8:45–10:45	**5.** Schools	Create a school (you become owner), list mine, switch school (C10)	MUST
10:45–13:30	**6.** Roles, permissions, members	Permission catalogue, roles, add/change/remove members (C11)	MUST core, SHOULD role editing
13:30–17:00	**7.** Frontend integration	Real sign up / sign in / create school in your React app; permission-based UI (C12)	MUST
17:00–19:00	**8.** Deploy	API on the droplet using the DigitalOcean database; frontend CI flags (C13)	SHOULD
19:00–20:00	**9.** Buffer	Acceptance checklist (C14), bug fixes, commit and tag	MUST

> **Be honest with yourself about the clock**
> You are new to Node.js. The estimates assume you do not get stuck for more than **30 minutes** on anything. If you do: read the Troubleshooting chapter, then ask (paste your code and the *exact* error). A one-hour stall costs you a whole SHOULD item. Commit to git after every checkpoint so you can always go back.

How each step is written

- **Goal** — what exists when the step is done. **Rules** — what it must do (this is the specification; write the code from it). **Hints** — the syntax you may not know. **Check** — commands with the answer you should see.
- Do the **Check** every time. Moving on with a broken step is the most expensive mistake.

- Rules about security (“same error for every failure”, “never return the hash”) are not optional decorations — they are the reason these routes can be trusted with a school’s data.

What changed since the previous guide

Topic	Now	Why
Which school a request acts in	Stored **inside the access token** (`tid`). No `X-Organization` header.	Your frontend sends only a Bearer token; this needs no change to its HTTP client.
Refresh token	An **httpOnly cookie named** `rt` , never in JSON. Now a MUST.	Your frontend’s `AuthProvider` already works this way, and a 15-minute token would log users out constantly otherwise.
Success / error JSON	Success: `{ "success": true, "data": ..., "meta": ... }` . Errors: `{ "message": "...", "errors": [{ "field", "message" }] }` .	That is what `apiClient` unwraps and what forms use for field errors.
Roles	Default roles per school use **your frontend’s role names**; permissions are `module.action` like `students.create` .	Your `RoleGuard` and `hasPermission()` already use these strings.
Database	PostgreSQL on **DigitalOcean** (plus an optional local one for development).	That is where your data will live.
Members and permissions	New chapter C11.	You asked for user permissions.

> **Never put secrets in chat, git or screenshots**
> Your database connection string contains a password. It goes only into `.env` on your machine and `/etc/karkaii/api.env` on the server — never into git, this guide’s examples, a screenshot or a message to me. If you ever paste one by accident, rotate that database password in DigitalOcean immediately.

Part A — Learn first (fast track)

You have about 2.5 hours for this part — read in this order

Section	Read now (★)	Skim or skip for now
A1 Node.js	★ One thread; project files; values/functions/arrays (**exercise 1**); modules; settings from the environment; **async code (exercise 4)**	Built-in modules (you will use `node:crypto` in C5)
A2 TypeScript	★ Basic types; object shapes; unions and "might be missing"; functions and Promises; reading errors	Generics, casts (read only the `unknown` box)
A3 Express	★ HTTP table; first server; request data; **middleware**; routers; **errors in Express 5**; zod; build a **reduced exercise 7** (GET, POST, PATCH; 40 minutes)	DELETE and the search parameter of the notes API
A4 Prisma	★ All of it — you use every query shown here in Part C	Exercise 8 (you do it for real in C2 and C6)
A5 Cookies & tokens	★ Short primer at the end of Part A	—

If a section is not clear after one read, move on and come back when Part C uses it — the build teaches by doing.

Everything in Part A is small and standalone. Make a folder `exercises/` in your project and put each exercise in its own file. Run any TypeScript file with `npx tsx exercises/01-basics.ts`.

A1. Node.js basics

What Node.js is

Node.js runs JavaScript outside the browser. A backend is just a Node program that starts a server, waits for requests, talks to a database and sends answers. **npm** is the tool that downloads libraries ("packages") for it. **TypeScript** is JavaScript with types added, which a tool (`tsx` while developing, `tsc` for production) turns into plain JavaScript.

One thread, never blocked

Node runs your code on **one thread**. That sounds weak, but a server spends most of its time *waiting* (for the database, the network, the disk). While one request waits, Node serves others. The rule that follows: **never run long synchronous work** (a huge loop, a blocking file read) inside a request, because everyone else waits. Waiting operations return a **Promise**, and you use `await` on them (see below).

Project files you will meet

File / folder	What it is
`package.json`	The project's identity card: name, the libraries it needs (`dependencies`), tools only for development (`devDependencies`) and shortcuts (`scripts`) you run with `npm run dev`.
`package-lock.json`	The exact versions installed. Commit it; never edit it.
`node_modules/`	Downloaded libraries. Huge; never commit (goes in `.gitignore`); recreate with `npm install`.
`.env`	Settings and secrets for *this machine* (database address, secret keys). Never commit.

`tsconfig.json`	How TypeScript should check and compile your code.

terminal: commands you will use all the time

```
npm init -y                 # create package.json
npm i express               # add a library (goes to "dependencies")
npm i -D typescript tsx     # add a development-only tool ("devDependencies")
npm install                 # install everything listed in package.json
npm run dev                 # run the "dev" script from package.json
npx tsx file.ts             # run a TypeScript file directly
```

Values, functions, arrays, objects

exercises/01-basics.ts

```ts
// variables: const = cannot be reassigned (use it by default), let = can be
const school = 'Greenfield';
let students = 120;
students = students + 5;

// template strings use backticks
console.log(`${school} has ${students} students`);

// functions (two ways to write the same thing)
function double(n: number): number {
  return n * 2;
}
const triple = (n: number): number => n * 3;
console.log(double(4), triple(4));

// arrays and the three methods you will use constantly
const marks = [45, 78, 92, 60];
const passed = marks.filter((m) => m >= 50);        // keep some
const percent = marks.map((m) => m / 100);          // transform each
const total = marks.reduce((sum, m) => sum + m, 0); // combine into one value
console.log(passed, percent, total);

// objects, destructuring, spread, optional chaining
const student: { name: string; class: string; address?: { city: string } } = { name: 'Asha', class: '8A' };
const { name } = student;                       // pull a field out
const moved = { ...student, class: '9A' };      // copy with a change
console.log(name, moved.class, student.address?.city);
// "?." stops safely (returns undefined) instead of crashing when something is missing: address is optional here
```

Do it: run it, then change the numbers, add a `find` call (`marks.find((m) => m > 90)`) and print the result.

Modules: splitting code into files

Every file is a module. Anything you want to use elsewhere must be **exported**, and the other file **imports** it. (The code you write uses `import` / `export`; the tooling converts it for Node.)

exercises/02-math.ts

```ts
export function add(a: number, b: number): number {
  return a + b;
}
export const PASS_MARK = 50;
```

exercises/02-main.ts

```ts
import { add, PASS_MARK } from './02-math';   // relative path, no .ts extension
console.log(add(2, 3), PASS_MARK);
```

Libraries are imported by name: `import express from 'express'` (a *default* export) or `import { z } from 'zod'` (a *named* export). The library's documentation tells you which.

Settings from the environment

`process.env` holds environment variables, always as **text or undefined**. In this project a `.env` file is loaded into it by the `dotenv` library.

exercises/03-env.ts

```
const port = Number(process.env.PORT ?? 3000);   // ?? = "use this if the left side is undefined"
console.log('port is', port, typeof port);
```

Try: `PORT=8080 npx tsx exercises/03-env.ts` and without it.

Async code: the most important idea in Node

Anything that waits (database, network, timers) is **asynchronous**: you start it, and get a **Promise** — a placeholder for a value that arrives later. `await` pauses *this function* (not the whole server) until the value is ready. You can only use `await` inside an `async` function.

exercises/04-async.ts

```
const sleep = (ms: number) => new Promise<void>((resolve) => setTimeout(resolve, ms));

async function fetchStudent(id: number) {
  await sleep(500);                       // pretend this is a database query
  if (id === 0) throw new Error('No such student');
  return { id, name: 'Student ' + id };
}

async function main() {
  console.log('start');

  const one = await fetchStudent(1);      // waits ~0.5 s
  console.log(one);

  // run things at the same time, wait for all of them
  const [a, b] = await Promise.all([fetchStudent(2), fetchStudent(3)]);
  console.log(a.name, b.name);            // total ~0.5 s, not 1 s

  // errors: a rejected promise becomes a thrown error at the await
  try {
    await fetchStudent(0);
  } catch (err) {
    console.log('caught:', err instanceof Error ? err.message : err);
  }
  console.log('end');
}

main();   // forgetting "await"/".catch" on a promise is a classic bug: the error disappears
```

> **Rules of thumb**
> (1) If a function returns a Promise, `await` it. (2) Wrap the *top* of a flow in try/catch (in Express 5 the framework does that for you — see A3). (3) Use `Promise.all` only when the calls do not depend on each other.

Built-in modules

Node ships with libraries you import with a `node:` prefix. You will use `node:crypto` (random tokens, hashes). Try it:

exercises/05-crypto.ts

```
import { createHash, randomBytes } from 'node:crypto';

const token = randomBytes(32).toString('base64url');          // a random, unguessable string
const hash = createHash('sha256').update(token).digest('hex'); // a one-way fingerprint of it
console.log(token, hash);
```

A2. TypeScript basics

TypeScript checks your code *before it runs*. Typos in field names, wrong argument types and forgotten "might be missing" cases become red underlines in your editor instead of crashes in production. It costs nothing at runtime: the types disappear when compiled.

Basic types and inference

exercises/06-types.ts

```
let name: string = 'Asha';
let age: number = 14;
let active: boolean = true;
let tags: string[] = ['a', 'b'];           // an array of strings
let pair: [string, number] = ['x', 1];     // a tuple: fixed length and order

const city = 'Chennai';   // you often don't need to write the type: TypeScript infers string
// age = 'fourteen';      // ERROR: string is not assignable to number  <- this is the point of TypeScript
```

Describing the shape of an object

Use an `interface` (or `type`; for objects they are nearly the same) to name a shape and reuse it.

shapes

```
interface Student {
  id: number;
  name: string;
  email?: string;           // ? = optional: the property may be missing
  readonly createdAt: Date; // cannot be changed after creation
}

function greet(s: Student): string {
  return 'Hello ' + s.name;
}
greet({ id: 1, name: 'Asha', createdAt: new Date() });   // ok
// greet({ id: 1 });   // ERROR: 'name' and 'createdAt' are missing
```

Unions, literal types and "might be missing"

A **union** (`A | B`) means "one of these". Literal types limit a value to specific strings. `null` / `undefined` are explicit — with `strict` mode on, TypeScript forces you to handle them. This one feature prevents a huge class of bugs.

unions

```
type Status = 'active' | 'disabled';          // only these two strings are allowed
let status: Status = 'active';
// status = 'banned';                          // ERROR

function lengthOf(value: string | undefined): number {
  // value.length                             // ERROR: value might be undefined
  if (value === undefined) return 0;          // "narrowing": after this check TypeScript knows it is a string
  return value.length;
}
const nickname: string | undefined = undefined;
const shown = nickname ?? 'no nickname';      // ?? gives a fallback for null/undefined
```

Functions, generics, Promises

functions

```
// parameter and return types
function percent(score: number, outOf = 100): number {   // outOf has a default value
  return (score / outOf) * 100;
}

// async functions return Promise<T>: "a promise of a T"
async function loadName(id: number): Promise<string> {
  return 'Student ' + id;
}

// generics: "works for any type T" - you will mostly READ these (Array<string>, Promise<number>)
function first<T>(items: T[]): T | undefined {
  return items[0];
}
const x = first([1, 2, 3]);        // x is number | undefined
const y = first(['a', 'b']);       // y is string | undefined
```

unknown, any, and casting

- `any` switches checking off. Avoid it. `unknown` means "I don't know yet" and forces you to check before use: request data and caught errors are `unknown`.
- A cast (`value as Type`) *tells* TypeScript to believe you. It does not check anything at runtime. Prefer validating with **zod** (Part C) so the type is proven, not claimed.

unknown

```
try {
  JSON.parse('{ not json');
} catch (err) {                    // err is "unknown"
  if (err instanceof Error) console.log(err.message);   // check, then use
}
```

Numbers vs bigint

Database ids in this project are `bigint` (very large whole numbers), written `123n` or `BigInt('123')`. Two things to remember: you cannot mix them with normal numbers (`1n + 1` is an error), and `JSON.stringify` cannot print a bigint — the backend converts ids to strings when sending JSON.

Type-only imports and reading errors

- `import type { AuthUser } from './x'` imports only a type (it vanishes at runtime). Use it when you import something purely for typing.
- **Reading a TypeScript error:** read the *last* line first — it says what was expected vs what you gave ("Type 'string' is not assignable to type 'number'"). The first lines are context.
- `strict: true` in `tsconfig.json` turns on the careful checks (null handling etc.). Keep it on.

Do it (exercise 6): in `06-types.ts` define an `interface Teacher` (name, subjects: string[], phone optional), write a function `describe(t: Teacher): string` that returns the name and the number of subjects, and call it. Then deliberately break a type and read the error.

A3. Express basics

HTTP in one minute

A client (browser, app, curl) sends a **request**; the server sends a **response**. A request has a **method** (what to do), a **path** (what to do it to), **headers** (extra info like the login token) and sometimes a **body** (JSON). A response has a **status code**, headers and a body.

Method	Meaning	Example
GET	read something	`GET /api/organizations`
POST	create something	`POST /api/auth/signup`
PATCH	change part of something	`PATCH /api/organizations/current`
PUT	replace something	`PUT /api/users/5/roles`
DELETE	remove something	`DELETE /api/users/5`

Status	Use it for
200 / 201 / 204	OK / Created (after a POST) / OK with no body
400	the request is malformed or invalid (validation failed)
401	not signed in, or the token is bad/expired (“who are you?”)
403	signed in, but not allowed (“I know you, but no”)
404	not found
409	conflict, e.g. the email or slug already exists
429	too many requests (rate limit)
500	a bug on the server — the client did nothing wrong

A first server

Express is a thin layer over Node’s HTTP server. Install: `npm i express@^5 && npm i -D @types/express@^5`.

exercises/07-server.ts

```ts
import express from 'express';

const app = express();
app.use(express.json());                    // parse JSON request bodies into req.body

app.get('/hello', (req, res) => {
  res.json({ message: 'hi' });              // sends JSON with status 200
});

app.listen(3000, () => console.log('http://localhost:3000'));
```

terminal

```
npx tsx exercises/07-server.ts       # in one terminal
curl -i http://localhost:3000/hello      # in another: -i also prints the status line and headers
```

Reading the request: params, query, body, headers

request data

```
app.get('/students/:id', (req, res) => {
  req.params.id;                       // /students/42        -> "42"   (always text)
  req.query.page;                      // /students/42?page=2 -> "2"    (text, or undefined)
  req.get('x-organization');           // a request header    -> text or undefined
  res.json({ id: req.params.id });
});

app.post('/students', (req, res) => {
  req.body;                            // the parsed JSON body (needs express.json())
  res.status(201).json({ created: true });   // choose the status, then send JSON
});
```

curl: sending JSON

```
curl -i -X POST http://localhost:3000/students \
  -H 'content-type: application/json' \
  -d '{"name":"Asha"}'
# without the content-type header the body is NOT parsed - the #1 beginner surprise
```

Middleware: the idea that makes Express click

A **middleware** is a function `(req, res, next)` that runs *before* your route. It can look at or change the request, then either **stop** (send a response, or throw) or call `next()` to pass the request on. Middleware runs in the **order you register it**.

middleware

```
// runs for every request
app.use((req, _res, next) => {
  console.log(req.method, req.url);
  next();                                     // continue to the next middleware / the route
});

// a middleware that can say no
function requireKey(req: express.Request, _res: express.Response, next: express.NextFunction) {
  if (req.get('x-key') !== 'secret') throw new Error('nope');   // stops here
  next();
}
app.get('/private', requireKey, (req, res) => res.json({ ok: true }));   // runs requireKey first
```

Order matters. Everything registered *after* `app.use(someMiddleware)` goes through it; everything registered before does not. In this project, that is exactly how login protection works: public routes are registered first, then the "authenticate" middleware, then everything else.

Routers: grouping routes

router

```
const router = express.Router();
router.get('/', (req, res) => res.json([]));          // GET  /notes
router.post('/', (req, res) => res.status(201).json({}));   // POST /notes
app.use('/notes', router);                            // mount the group under a path
```

Errors in Express 5

In Express 5, if a route handler **throws** (or an `async` handler rejects), Express passes the error to the **error handler**: a middleware with **four** arguments, registered **last**. So handlers just throw; one place decides the response.

error handling

```
app.get('/boom', async () => {
  throw new Error('something broke');
});

app.use((err: unknown, _req: express.Request, res: express.Response, _next: express.NextFunction) => {
  console.error(err);                                  // log the details for yourself
  res.status(500).json({ message: 'Internal server error' });   // tell the client nothing sensitive
});
```

Exercise 7: a mini notes API (in memory)

Build it in `exercises/07-notes.ts`. No database: keep notes in an array. This exercise uses every Express idea you need.

Route	Behaviour
`GET /notes`	200 + the array of notes. Optional `?search=text` keeps notes whose title contains it (case-insensitive).
`POST /notes`	Body `{ "title": "...", "done": false }`. Title is required (2–100 chars). 201 + the created note with a new numeric `id`. 400 if invalid.
`GET /notes/:id`	200 + the note, or 404 `{"message":"Note not found"}`. Non-numeric id → 400.
`PATCH /notes/:id`	Change `title` and/or `done`. 404 if missing.
`DELETE /notes/:id`	204, or 404.

Requirements that practise the real thing: (1) put the routes in an `express.Router()` mounted at `/notes`; (2) write a logging middleware; (3) write one error-handling middleware and a small `HttpError` class (`status` + `message`) so handlers can `throw new HttpError(404, 'Note not found')`; (4) validate input with zod (next box); (5) test every route with curl, including the failures.

> **zod in 2 minutes (validation)**
>
> Never trust request data. A **zod** schema describes valid input; `schema.parse(data)` returns clean, correctly-typed data or **throws** a `ZodError` (which your error handler turns into a 400).
>
> zod
>
> ```
> import { z } from 'zod';
>
> const createNoteSchema = z.object({
>   title: z.string().trim().min(2).max(100),
>   done: z.boolean().default(false),
> }).strict();                                // .strict(): unknown extra fields are rejected
>
> type CreateNote = z.infer<typeof createNoteSchema>;   // the TypeScript type, generated from the schema
>
> const input = createNoteSchema.parse(req.body);       // throws ZodError if invalid
> // in the error handler:  if (err instanceof ZodError) -> res.status(400).json({ issues: err.issues })
>
> // query strings are text, so convert numbers with coerce:
> z.object({ page: z.coerce.number().int().min(1).default(1) }).parse(req.query);
> ```

A4. Databases and Prisma basics

Relational databases in five ideas

Idea	Meaning
Table / row / column	A table (`users`) holds rows (one per person) made of columns (`email` , `full_name`).
Primary key	A column that uniquely identifies a row (`id`).
Foreign key	A column pointing at another table's row (`memberships.user_id` → `users.id`). The database refuses a value that points nowhere.
Unique constraint	The database refuses a second row with the same value (e.g. `users.email`). This is safer than checking in code first, because two simultaneous requests can both pass a check-first.
Transaction	A group of changes that all succeed or all roll back — creating a school touches six tables and must never half-finish.

PostgreSQL is the database. You rarely write SQL by hand here: **Prisma** is an ORM — you describe tables in a `schema.prisma` file, Prisma creates them with a **migration** (a saved, versioned SQL script), and gives you a typed client to query them.

The Prisma workflow

terminal

```
npm i @prisma/client@^6 && npm i -D prisma@^6
# 1. write / change prisma/schema.prisma
npx prisma migrate dev --name what_you_changed   # 2. creates the SQL migration, applies it, regenerates the client
npx prisma studio                                # 3. a browser UI to look at your tables and rows
npx prisma migrate reset                         # wipes the dev database and re-applies everything (dev only!)
```

A schema example

prisma/schema.prisma (example, not your project)

```
model School {
  id       BigInt    @id @default(autoincrement())   // primary key, counts 1, 2, 3...
  slug     String    @unique                         // unique constraint
  name     String
  createdAt DateTime @default(now()) @map("created_at")   // @map: real column name in the database
  students Student[]                                 // the "other side" of the relation (no column)

  @@map("schools")                                   // real table name
}

model Student {
  id       BigInt  @id @default(autoincrement())
  schoolId BigInt  @map("school_id")
  email    String
  school   School  @relation(fields: [schoolId], references: [id])   // the foreign key

  @@unique([schoolId, email])                        // unique TOGETHER: an email once per school
  @@map("students")
}
```

Querying

queries (example)

```ts
import { PrismaClient } from '@prisma/client';
const prisma = new PrismaClient();

// create
const school = await prisma.school.create({ data: { slug: 'greenfield', name: 'Greenfield' } });

// read one / many  (findFirst returns null when nothing matches)
const one = await prisma.school.findFirst({ where: { slug: 'greenfield' } });
const many = await prisma.student.findMany({
  where: { schoolId: school.id, email: { contains: 'a', mode: 'insensitive' } },
  orderBy: { id: 'asc' },
  take: 20, skip: 0,
});

// choose fields (select) or include relations
const withStudents = await prisma.school.findFirst({
  where: { slug: 'greenfield' },
  select: { id: true, name: true, students: { select: { email: true } } },
});

// update: updateMany lets you filter by any column and tells you how many rows matched
const { count } = await prisma.student.updateMany({
  where: { id: 5n, schoolId: school.id },      // 5n is a bigint literal
  data: { email: 'new@example.com' },          // a field set to undefined is left unchanged
});

// nested create: create a row AND related rows together
await prisma.school.create({
  data: { slug: 'riverdale', name: 'Riverdale', students: { create: [{ email: 'a@x.com' }] } },
});

// transaction: use "tx" inside, not "prisma"
await prisma.$transaction(async (tx) => {
  const s = await tx.school.create({ data: { slug: 'sunrise', name: 'Sunrise' } });
  await tx.student.create({ data: { schoolId: s.id, email: 'b@x.com' } });
});   // if anything inside throws, everything is rolled back
```

Catching a unique-constraint error

P2002

```ts
import { Prisma } from '@prisma/client';

try {
  await prisma.school.create({ data: { slug: 'greenfield', name: 'Again' } });
} catch (e) {
  if (e instanceof Prisma.PrismaClientKnownRequestError && e.code === 'P2002') {
    // P2002 = "unique constraint failed": answer with a 409 instead of a 500
  } else throw e;
}
```

Do it (exercise 8): after C1 and C2 you will have a database. Until then, if you want to try Prisma early: create a tiny schema with `School` and `Student` like the example, run a migration, write `exercises/08-prisma.ts` that creates a school with two students, lists them, tries to create a duplicate slug and catches the error. Look at the rows in Prisma Studio.

A5. Cookies, tokens and password hashing in five minutes

Part C leans on three small ideas. Read them now so C5 feels familiar.

Cookies

A **cookie** is a small value the server asks the browser to store (`Set-Cookie` response header). The browser then sends it back automatically (`Cookie` request header) on matching requests. Attributes control it:

Attribute	Meaning
`httpOnly`	JavaScript on the page cannot read it (protects against script-injection theft). The browser still sends it.
`secure`	Only sent over HTTPS. (Browsers drop it on plain http — the classic "login works then logs me out" bug.)
`sameSite`	`lax` : not sent on cross-site requests triggered by other sites (CSRF protection).
`path`	Only sent to URLs under this path (we use `/api/auth`).
`maxAge`	Lifetime in milliseconds.

cookies in Express (needs the cookie-parser middleware)

```
import cookieParser from 'cookie-parser';
app.use(cookieParser());                          // now req.cookies exists

res.cookie('rt', value, {                         // send a cookie
  httpOnly: true, secure: false, sameSite: 'lax', path: '/api/auth', maxAge: 30 * 24 * 60 * 60 * 1000,
});
const token = req.cookies.rt;                     // read it (string | undefined)
res.clearCookie('rt', { path: '/api/auth' });     // delete it: the path MUST match
```

JWT (JSON Web Token)

A JWT is a small signed text: `header.payload.signature` . The payload (e.g. `{ sub: "5", tid: "3" }`) is only *encoded*, not encrypted — anyone can read it, so never put secrets in it. The **signature**, made with your secret, proves nobody edited it. `jwt.verify` checks the signature and the expiry and throws if either is wrong.

jsonwebtoken

```
import jwt from 'jsonwebtoken';

const token = jwt.sign({ sub: '5', tid: '3' }, SECRET, { algorithm: 'HS256', expiresIn: 900 });   // 900 seconds
try {
  const payload = jwt.verify(token, SECRET, { algorithms: ['HS256'] });   // string | JwtPayload
} catch {
  // bad signature or expired -> answer 401
}
```

Password hashing

argon2

```
import * as argon2 from 'argon2';

const hash = await argon2.hash('my password');          // store this, never the password
const ok = await argon2.verify(hash, 'my password');    // true / false
```

Do it (exercise 9, 15 minutes): in `exercises/09-tokens.ts` hash a password and verify a right and a wrong one; sign a token with a 2-second expiry, verify it immediately, wait 3 seconds with `await sleep(3000)` , verify again and catch the error; then decode the payload by hand: `JSON.parse(Buffer.from(token.split('.')[1], 'base64url').toString())` — and see that it is readable.

Part B — The design

Ten minutes of reading that saves hours. Every rule in Part C comes from here.

B1. Many schools, one system

Karkaii serves many schools from **one** application and **one** database. A school is a **tenant** (the ERD says `organization`; in code we say *organization*). The promise of the whole system: *a user of school A can never see, change or even detect the data of school B.* We use shared tables with a `tenant_id` column (your ERD's design: cheap, one migration).

> **The one rule.** The school a request acts in is **verified against the caller's membership on every request** and then used in **every** database query as `tenantId`. It is never taken, unchecked, from the URL, query string or body.

B2. One person, many schools — memberships

A correspondent may run three schools; a parent may have children in two; a teacher may work at two. They must have **one login**. So there are three separate things:

User
one per PERSON
email, password
(no school inside)

↔

Membership
one per PERSON + SCHOOL
"Asha belongs to Greenfield"

→

Roles of that membership
what Asha is INSIDE Greenfield

Person (one account)	School	Role there
Asha — asha@example.com	Greenfield School	owner
	Riverdale School	owner (same role in a second school)
	Sunrise School	teacher (a different role in a third)
Ravi — ravi@example.com	Greenfield School	accountant

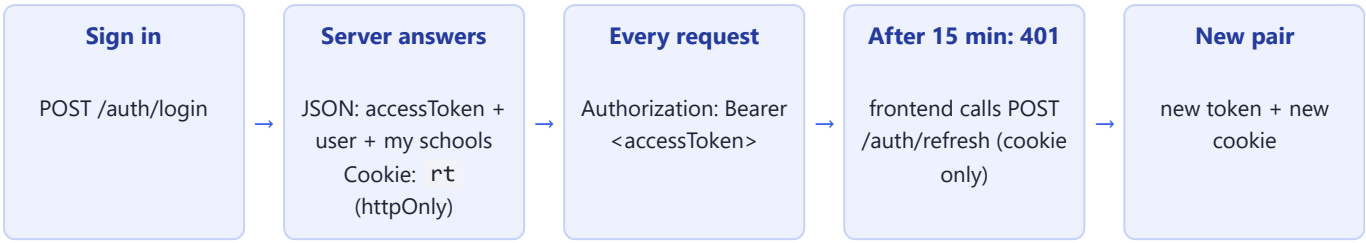
- `users`: no `tenant_id`; `email` unique across the whole system.
- `memberships`: unique (tenant, user), with a `status` (active / suspended).
- `roles`: belong to **one school** (each school gets its own copies). `membership_roles`: which roles a membership has.
- Permissions are always evaluated **inside one school**.

What changes in your ERD

ERD today	Change
`users.tenant_id`	Remove. Users are global; keep email globally unique.
(nothing)	Add `memberships`.
`user_roles`	Becomes `membership_roles` (membership_id, role_id).
`refresh_tokens.tenant_id`	Keep but make it **optional**: it records which school this login session is currently acting in.
`audit_log.tenant_id`	Make it optional.

B3. Sessions: access token, refresh cookie, active school

This is the part that must match your frontend exactly. Read it twice.



- **Access token** = a JWT containing the person id (`sub`) and the **active school id** (`tid`, absent if the person has no school yet). Lives 15 minutes. On *every* request the server re-checks the database: the person is active, and (if `tid` is present) the membership and school are still active. So suspending someone takes effect immediately.
- **Refresh token** = a long random string in an **httpOnly cookie named** `rt` (JavaScript cannot read it, so XSS cannot steal it). Only its SHA-256 hash is stored. It is **single-use**: each refresh returns a new cookie. The cookie's path is `/api/auth` so the browser sends it only to auth routes.
- **Active school**: after sign in you act in your first school (or the one you named); `POST /auth/switch-organization` changes it and gives a new token. The session remembers the active school, so refreshing keeps it.
- **Theft detection**: if an already-used refresh token appears again *much later*, the whole login ("family") is revoked. **But** if it appears within **10 seconds** it is almost certainly a second browser tab or a React double-start racing — your frontend expects a **409** in that case and quietly retries. So: used ≤ 10 s ago → 409 (revoke nothing); used longer ago → revoke the family, 401.

Passwords

Store only an **argon2** hash (slow and memory-hungry on purpose). Never log or return it. Same error and same response time for "unknown email" and "wrong password".

B4. Roles and permissions

A **permission** is a code `<module>.<action>`, e.g. `students.create`. Your Roles & Permissions screen already uses **ten actions** and these modules, so the catalogue matches it:

Actions (10)	`view, create, edit, delete, import, export, approve, print, download, manage`
Frontend modules (8)	`dashboard, students, admissions, teachers, academics, attendance, examinations, finance`
Backend modules (3)	`organization, members, roles`

11 modules × 10 actions = **110 permissions**, generated by a loop (do not type them by hand). A **role** is a bundle of permissions; roles belong to one school. Routes ask for permissions, never role names, so schools can later create their own roles.

Default roles every new school receives

Role code	Permissions
`owner`	**Everything, always** (implemented as a wildcard in code, so new permissions apply automatically). The creator of the school. Also the only role that can assign `owner`.
`admin`	All permissions except `organization.manage`

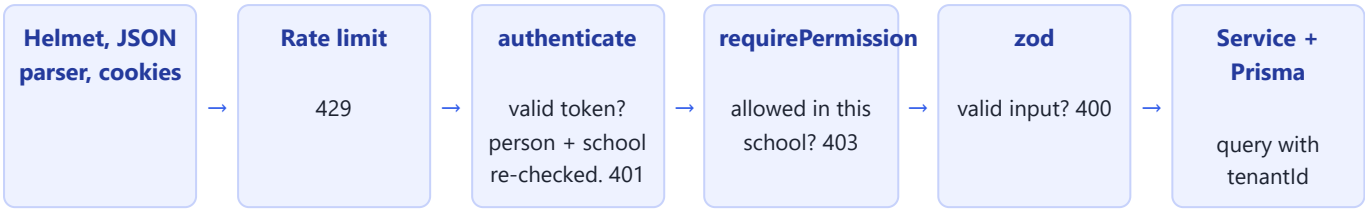
`teacher`	`dashboard.view` ; `students.view` ; `attendance.view/create/edit` ; `academics.view` ; `examinations.view/create/edit`
`accountant`	`dashboard.view` ; `students.view` ; all ten `finance.*`
`receptionist`	`dashboard.view` ; `students.view` ; `admissions.view/create/edit`
`student` , `parent` , `librarian` , `warden` , `transport_manager` , `nurse` , `security`	`dashboard.view` only (to start)

These role codes are exactly the role names your frontend knows (`USER_ROLE` in `app.constants.ts`), except `owner` . The frontend has one `role` per user to choose a home dashboard, so the backend sends a **primary role** computed from the person's roles in the active school: `owner` or `admin` → `admin` ; otherwise the first match in the order teacher, accountant, librarian, receptionist, warden, transport_manager, nurse, security, parent, student; a custom role the frontend does not know → `receptionist` (a neutral staff dashboard). The full permission list travels separately as `permissions` (owners get `["*"]`).

B5. What your frontend expects (the contract)

Frontend behaviour (from its code)	What the backend must do
API base is `/api` ; the dev server proxies `/api` to `localhost:5000` (`API_PROXY_TARGET`)	Listen on port 5000, all routes under `/api` . Same origin → no CORS, cookies just work.
Every request uses `credentials: 'include'`	Cookie-based refresh works as designed in B3.
`unwrapEnvelope` : if the body has `success` and `data` , returns `data`	Wrap every success body: `{ success: true, data, meta? }` . Lists put paging in `meta: { page, limit, total, pages }` . 204 has no body.
Errors: reads `message` and `errors: [{ field, message }]` ; 400 with `errors` = form validation	Validation failures: 400 `{ message, errors: [{ field, message }] }` . Others: `{ message }` .
Login: `POST /auth/login` `{email,password}` → `{ accessToken, user }` ; `user` = `{ id, name, email, role, avatar, phone }`	Return those fields (plus `permissions` , `organization` , and `organizations`). `name` is the full name.
Refresh: `POST /auth/refresh` , no body, cookie `rt` ; same response as login. **409** = "just rotated, retry"	Exactly as in B3.
Logout: `POST /auth/logout`	Revoke the session, clear the cookie, 204.
`GET /auth/me` → the user object	Same `user` shape.
List endpoints use `page` , `limit` (max 200), `q` (search), `sort=-field`	Support at least `page` , `limit` , `q` on the members list.
Mock vs real is **one global flag** (`VITE_USE_MOCK_API`)	Frontend work in C12: give auth its own switch so students/dashboard stay on mocks.

B6. What happens to one request



Protected by default: in `app.ts` the few public routes (health, signup, login, refresh, logout) are registered *before* the `authenticate` middleware; everything after it needs a valid token.

Part C — Build it

Suggested folder layout (you create the files as the steps ask):

```
project layout

src/
  server.ts                starts the HTTP server
  app.ts                   builds the Express app (middleware ORDER lives here)
  config.ts   db.ts        settings from .env; the Prisma client
  permissions.ts           permission catalogue + default roles (data)
  lib/                     errors.ts  bigint.ts  respond.ts  hash.ts  tenant.ts
  middleware/              authenticate.ts  requirePermission.ts  errorHandler.ts
  modules/
    auth/                  auth.schemas.ts  session.ts  auth.service.ts  auth.routes.ts
    organizations/         organizations.schemas.ts  .service.ts  .routes.ts
    members/               members.schemas.ts  .service.ts  .routes.ts
    roles/                 roles.schemas.ts  .service.ts  .routes.ts
  types/express.d.ts       tells TypeScript about req.user and req.tenant
prisma/                    schema.prisma  seed.ts  migrations/
exercises/                 your Part A exercises
```

Why services? Route files only translate HTTP (read the request, call a function, send the response). The real logic lives in `*.service.ts` as plain functions.

C0. The DigitalOcean database (45 min)

Goal: a database and a limited user on your DigitalOcean cluster, reachable from the droplet (and from your laptop for one-off commands). MUST

> **Read before you click**
>
> The cluster in your screenshot is called **student-360-test-cluster** (PostgreSQL 18, 1 GB RAM, 1 vCPU, 10 GiB, Bangalore). The name suggests it belongs to another project. **Do not touch its existing databases** (especially `defaultdb`). You will add a *new, separate* database and user inside it. A 1 GB cluster shared by two projects is tight; plan a dedicated cluster for production later.

C0.1 Create the database and the user

1. DigitalOcean → **Databases** → open the cluster → tab **Users & Databases**.
2. **Databases** section: add a database named `karkaii`.
3. **Users** section: add a user named `karkaii_app`. **Copy its password immediately** (store it in your password manager).

C0.2 Trusted sources (the firewall)

1. Cluster → tab **Settings** → **Trusted sources** → edit.
2. Add your **droplet** (the one at `64.227.129.36`) and **your current IP** (there is an "add current IP" button).
3. If a connection later hangs and times out, your home IP changed — add the new one. (If no trusted sources are listed at all, the database accepts connections from anywhere: always keep at least the droplet listed.)

C0.3 Give `karkaii_app` the right to create tables

New users have no rights on a new database. As the admin user (`doadmin`), grant them. Get the admin connection string from the cluster's **Overview** tab ("Connection details"; choose user `doadmin`, database `karkaii`). You need the `psql` client. The command below runs it through Docker, so you don't have to install anything:

terminal

```
# paste the doadmin connection string between the quotes (it contains the admin password: do not save it in a file or in shell history files you share)
docker run --rm -it postgres:16 psql "postgresql://doadmin:ADMIN_PASSWORD@HOST:25060/karkaii?sslmode=require"
```

inside psql

```
GRANT ALL PRIVILEGES ON DATABASE karkaii TO karkaii_app;
GRANT ALL ON SCHEMA public TO karkaii_app;
\q
```

(PostgreSQL 15 and newer no longer let ordinary users create objects in the `public` schema; the second line fixes that.) If a command says "permission denied", copy the exact message to me — as a fallback you can use `doadmin` temporarily.

C0.4 The connection string for the app

Overview → Connection details → user `karkaii_app`, database `karkaii` → **Connection string**. It looks like `postgresql://karkaii_app:PASSWORD@HOST:25060/karkaii?sslmode=require`. Add two options for Prisma:

the shape of your DigitalOcean URL

```
postgresql://karkaii_app:PASSWORD@HOST:25060/karkaii?sslmode=require&connection_limit=5
```

- `connection_limit=5`: small clusters allow only a few connections in total. Keep Prisma's pool small.
- **TLS error?** DigitalOcean uses its own certificate authority. If Prisma reports a certificate / TLS error (`P1011`, "self-signed certificate"), either (quick) add `&sslaccept=accept_invalid_certs`, or (better) download the cluster's **CA certificate** from the Overview tab and add `&sslcert=/full/path/ca-certificate.crt`. This guide was not tested against your cluster, so tell me the exact error if neither works.
- The cluster runs PostgreSQL **18**. Prisma 6 uses standard features, so it should work, but I could not verify that version. The first `migrate deploy` in C13 is the test.
- If the droplet is in the same region (Bangalore) as the cluster, DigitalOcean also offers a **private (VPC) hostname** — faster and not exposed to the internet. Use it on the droplet if shown.

> **Where this URL goes**
>
> **Never** into git, never into a screenshot. Two places only: `.env.do` on your laptop (git-ignored; used for one-off commands) and `/etc/karkaii/api.env` on the droplet (C13).

C0.5 A local database for development (recommended)

Develop against a **local** PostgreSQL and use DigitalOcean for the deployed app. Why: `prisma migrate dev` needs to create a temporary "shadow" database, which a limited DigitalOcean user cannot do; your laptop IP changes; and a local database is faster and free. Prisma migrations created locally are applied to DigitalOcean with `migrate deploy`.

terminal

```
docker run -d --name karkaii-pg \
  -e POSTGRES_USER=karkaii -e POSTGRES_PASSWORD=karkaii_dev -e POSTGRES_DB=karkaii \
  -p 5432:5432 -v karkaii_pgdata:/var/lib/postgresql/data postgres:16
# next time:  docker start karkaii-pg
```

> **Check**
> `docker ps` shows `karkaii-pg`. The DigitalOcean checks come later (C13), but you can test the connection now: `docker run --rm postgres:16 psql "YOUR_DO_URL_WITHOUT_the_connection_limit_part" -c "select version();"` should print PostgreSQL 18.

C1. Project setup

Goal: a TypeScript project that runs. MUST — **if you were given the starter zip**, unzip it, run `npm install`, then run **one extra install** (this guide needs it and the zip predates it): `npm i cookie-parser && npm i -D @types/cookie-parser`. Then do steps 3–5 only (the zip already has steps 1–2).

1. Tools: Node 22 LTS (`node -v`; `nvm install 22` if yours is newer — argon2 is tested on 22) and Git.
2. Create the project (inside WSL) and install:

terminal

```
mkdir -p ~/karkaii/backend && cd ~/karkaii/backend
npm init -y
npm i express@^5 @prisma/client@^6 argon2 cors cookie-parser dotenv@^16 express-rate-limit@^7 helmet
jsonwebtoken@^9 zod@^3
npm i -D prisma@^6 typescript@^5 tsx @types/express@^5 @types/node@^22 @types/jsonwebtoken@^9 @types/cors@^2
@types/cookie-parser
```

Edit `package.json`: replace `"scripts"` and add the `prisma` block:

package.json (these two blocks)

```
"scripts": {
  "dev": "tsx watch src/server.ts",
  "build": "tsc -p tsconfig.build.json",
  "start": "node dist/server.js",
  "typecheck": "tsc --noEmit"
},
"prisma": { "seed": "tsx prisma/seed.ts" },
```

tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022", "module": "commonjs", "moduleResolution": "node", "lib": ["ES2022"],
    "strict": true, "esModuleInterop": true, "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true, "resolveJsonModule": true, "noEmit": true
  },
  "include": ["src", "prisma"]
}
```

tsconfig.build.json

```
{
  "extends": "./tsconfig.json",
  "compilerOptions": { "noEmit": false, "outDir": "dist", "rootDir": "src" },
  "include": ["src"]
}
```

3. **Protect secrets first:** `.gitignore` before your first commit.

.gitignore

```
node_modules/
dist/
```

```
.env
.env.*
.pgdata/
*.log
```

4. Create `.env` for **local** development (`openssl rand -hex 32` makes the secret) and `.env.do` for DigitalOcean one-off commands:

   .env

   ```
   DATABASE_URL="postgresql://karkaii:karkaii_dev@localhost:5432/karkaii?schema=public"
   JWT_ACCESS_SECRET=paste-the-64-character-secret-here
   JWT_ACCESS_TTL=900
   REFRESH_TTL_DAYS=30
   AUTH_RATE_LIMIT=10
   COOKIE_SECURE=false
   PORT=5000
   ```

   .env.do (never commit; only the database URL)

   ```
   DATABASE_URL="postgresql://karkaii_app:PASSWORD@HOST:25060/karkaii?sslmode=require&connection_limit=5"
   ```

   `COOKIE_SECURE=false` is for plain-http local development; production over HTTPS uses `true` (C13). Port 5000 and the `/api` prefix match your frontend's dev proxy.

5. **Git:** `git init -b main && git add -A && git commit -m "Project setup"`. Check `git status` never lists `.env`. Create a **new empty** GitHub repo for the backend whenever you like (for example `CreatixHub/karkaii-backend`) and push; commit after every checkpoint from now on.

> **Check**
> `git status` is clean and does not mention `.env` or `.env.do`. `npx tsc --version` prints 5.x.

C2. The data model

Goal: nine tables, created by a Prisma migration on your **local** database. MUST

Rules: write `prisma/schema.prisma` yourself from this specification (Part A4 shows every syntax you need). Ids are `BigInt @id @default(autoincrement())`. Use `camelCase` field names with `@map("snake_case")` and `@@map("table_name")`. Timestamps are `DateTime` with `@db.Timestamptz(6)`.

Table	Columns (notes)	Constraints
`organizations`	id; slug; name; timezone (default "UTC"); status (default "active": active | suspended); created_at (default now); updated_at (auto); deleted_at (optional)	slug unique
`users`	id; email (stored lower-case); full_name; password_hash; phone (optional); avatar_url (optional); status (default "active": active | disabled); failed_login_count (int, default 0); locked_until (optional); last_login_at (optional); password_changed_at (optional); email_verified_at (optional); created_at; updated_at; deleted_at (optional); version (int, default 1)	email unique (global)
`memberships`	id; tenant_id → organizations; user_id → users (cascade); status (default "active": active | suspended); created_at; updated_at	unique (tenant_id, user_id); index user_id
`roles`	id; tenant_id → organizations; code; name; description (optional); is_system (bool, default false); created_at; updated_at	unique (tenant_id, code)

`permissions`	id; code; module; action; description (optional); created_at	code unique
`role_permissions`	role_id → roles (cascade); permission_id → permissions; tenant_id → organizations; created_at	primary key (role_id, permission_id)
`membership_roles`	membership_id → memberships (cascade); role_id → roles (cascade); tenant_id → organizations; created_at	primary key (membership_id, role_id)
`refresh_tokens`	id; user_id → users (cascade); **tenant_id (optional)** → organizations (the school this login is acting in); family_id (uuid); token_hash; user_agent (optional); ip (optional text); expires_at; used_at (optional); revoked_at (optional); revoked_reason (optional); created_at	token_hash unique; index family_id and user_id
`audit_log`	id; tenant_id (optional) → organizations; actor_id (optional); action; entity_type; entity_id (optional text); ip (optional); created_at	index (tenant_id, created_at)

Hints: (1) A relation needs two sides: the field with `@relation(fields: [...], references: [...])` and an array field on the other model (`memberships Membership[]`); Prisma's error message names the missing one. (2) Composite primary key: `@@id([roleId, permissionId])` . (3) uuid: `String @db.Uuid` . (4) Optional field: `tenantId BigInt? @map("tenant_id")` with an optional relation (`Organization?`). (5) `@updatedAt` fills the update time.

```terminal
npx prisma format                      # checks the schema, shows errors with line numbers
npx prisma migrate dev --name init      # creates the tables in your LOCAL database
npx prisma studio                       # look at them in the browser
```

> **Check**
>
> Prisma Studio shows 9 empty tables. In `memberships` try to insert two rows with the same tenant and user — the database must refuse. Commit `prisma/migrations/` .

C3. Foundations

Goal: the small pieces every route uses. MUST

File	What it must do
`src/config.ts`	Load `.env` (`import 'dotenv/config'`), validate with a **zod schema**, export a typed `config` : `DATABASE_URL` ; `JWT_ACCESS_SECRET` (32+ characters); `JWT_ACCESS_TTL` (default 900); `REFRESH_TTL_DAYS` (default 30); `AUTH_RATE_LIMIT` (default 10); `COOKIE_SECURE` (boolean, default false); `PORT` (default 5000); `CORS_ORIGINS` (optional). Invalid: print each problem and `process.exit(1)` . *Hints:* `schema.safeParse(process.env)` ; `z.coerce.number()` because env values are text; for the boolean use `z.enum(['true','false']).default('false').transform(v => v === 'true')` .
`src/db.ts`	Create and export **one** `PrismaClient` .
`src/lib/errors.ts`	`class HttpError extends Error` with `status` , a message, and an optional `errors` array of `{ field, message }` . Anywhere: `throw new HttpError(409, 'Email already used')` .
`src/lib/bigint.ts`	(1) Make BigInt JSON-serializable: give `BigInt.prototype` a `toJSON` returning `this.toString()` (cast: `(BigInt.prototype as unknown as { toJSON: () => string }).toJSON = function (this: bigint) { return this.toString(); };`). (2) `parseId(value: unknown): bigint` — digit strings only (`/^\d{1,18}$/`), else `HttpError(400, 'Invalid id')` .
`src/lib/respond.ts`	The frontend expects `{ success, data, meta }` . Write tiny helpers: `ok(res, data, meta?)` → status 200; `created(res, data)` → 201; `noContent(res)` → 204 with no body. Each (except 204) sends `{ success: true, data, meta }` .
`src/middleware/errorHandler.ts`	An Express error handler (4 parameters, type `ErrorRequestHandler`). `ZodError` → 400 `{ message: 'Validation failed', errors: [{ field: path.join('.'), message }] }` . `HttpError` → its status and `{ message, errors? }` . Bad JSON (`SyntaxError` with a `body` property) → 400. Anything else → `console.error` it, send 500 `{ message: 'Internal server error' }` (never leak details). Note: the frontend reads `errors` , not `issues` .

`src/types/express.d.ts`	Teach TypeScript that `req.user` and `req.tenant` exist (declaration merging):
	pattern
	`import type { AuthUser } from '../middleware/authenticate';`
	`import type { TenantContext } from '../lib/tenant';`
	
	`declare global {`
	`  namespace Express {`
	`  interface Request {`
	`  user?: AuthUser;`
	`  tenant?: TenantContext;`
	`  }`
	`  }`
	`}`
	`export {};`
	
	It shows errors until you write those two types in C5 and C8 — fine.

Check

Start with `JWT_ACCESS_SECRET=abc` : your app must refuse to start and name the variable. Restore it.

C4. A skeleton app you will grow

Goal: a running server with the final middleware **order**, so every later step just adds a line. MUST

Build `createApp()` in `src/app.ts` (exported) and start it in `src/server.ts`. Order is the security design; in `createApp()` :

1. `app.set('trust proxy', 1)` and `app.disable('x-powered-by')` .
2. `import './lib/bigint'` (so ids become strings in JSON), `app.use(helmet())` , `app.use(cors({ origin: config.CORS_ORIGINS?.split(',') ?? false }))` , `app.use(express.json({ limit: '100kb' }))` , `app.use(cookieParser())` (this makes `req.cookies.rt` available).
3. A global rate limit: `rateLimit({ windowMs: 60_000, limit: 100, standardHeaders: true, legacyHeaders: false })` .
4. **Public routes**: `GET /api/health` → run `prisma.$queryRaw` with `SELECT 1` , then `ok(res, { status: 'ok' })` .
5. (Later steps add here: `app.use('/api/auth', publicAuthRouter)` .)
6. `app.use('/api', authenticate)` — **the dividing line** (added in C8). Everything after it needs a signed-in person.
7. (Later: protected routers.) Then a 404 (`app.use((_req, _res, next) => next(new HttpError(404, 'Not found')))`) and, **last of all**, `app.use(errorHandler)` .

`server.ts` : `createApp().listen(config.PORT, ...)` , plus clean shutdown on `SIGINT` / `SIGTERM` (`server.close(...)` then `prisma.$disconnect()`).

Check

terminal

```
npm run dev                                # leave running; it restarts on every save
curl -i http://localhost:5000/api/health   # 200 {"success":true,"data":{"status":"ok"}}
curl -i http://localhost:5000/api/nothing  # 404 {"message":"Not found"}
curl -i -X POST http://localhost:5000/api/health -H 'content-type: application/json' -d '{bad'   # 400
Invalid JSON (or 404) - never an HTML error page
```

Stop the database (`docker stop karkaii-pg`) and call `/api/health` : you must get a clean 500 JSON, not a crash. Start it again.

C5. The session engine

Goal: the shared machinery for tokens, cookies and the user JSON your frontend expects. Sign up, sign in, refresh and switch-school all reuse it. MUST

5a. `src/lib/hash.ts` and `src/lib/tenant.ts`

`hash.ts` : export `sha256(value: string): string` using `createHash('sha256').update(value).digest('hex')` from `node:crypto` (Part A1, exercise 5). Refresh tokens are stored only as this hash.

`tenant.ts` holds everything about "a person acting in a school":

Export	What it must do
`interface TenantContext`	`{ id: bigint; slug: string; name: string; roles: string[]; permissions: Set<string> }` — the caller's role codes and permission codes **in that school**.
`loadTenantContext(userId, tenantId)`	Returns a `TenantContext` or `null`. One Prisma query on `membership`: `where: { userId, tenantId, status: 'active', tenant: { is: { status: 'active', deletedAt: null } } }`; select the tenant (`id, slug, name`) and for each role its code and permission codes (`roles: { select: { role: { select: { code: true, rolePermissions: { select: { permission: { select: { code: true } } } } } } } } } } }`). Flatten into role codes and a `Set` of permission codes. **If a role code is `owner`, add the wildcard `'*'` to the set.**
`hasPermission(tenant, code)`	`tenant.permissions.has('*') || tenant.permissions.has(code)`
`listMyOrganizations(userId)`	All **active** memberships of the person whose school is active and not deleted, ordered by membership id: `[{ id, slug, name, roles: ['owner'] }]`.
`pickTenantId(userId, preferredSlug?)`	The school to act in right now: the one named by `preferredSlug` if the person is an active member, otherwise their **first** active membership, otherwise `null`.

5b. `src/modules/auth/session.ts`

Export	What it must do
`signAccessToken(userId, tenantId | null)`	`jwt.sign({ sub: String(userId), tid: String(tenantId) }, config.JWT_ACCESS_SECRET, { algorithm: 'HS256', expiresIn: config.JWT_ACCESS_TTL })`. Leave `tid` out when there is no school. (`expiresIn` as a **number** of seconds.)
`createSession({ userId, tenantId, meta, familyId? })`	Makes a refresh token (`randomBytes(48).toString('base64url')`), inserts a `refresh_tokens` row (`tokenHash: sha256(token)`, `familyId: familyId ?? randomUUID()`, `tenantId`, `expiresAt` = now + `REFRESH_TTL_DAYS` days, user agent and ip from `meta`), and returns `{ accessToken, refreshToken }`.
`setRefreshCookie(res, token)` / `clearRefreshCookie(res)`	The cookie must be exactly: name `rt`, `httpOnly: true`, `secure: config.COOKIE_SECURE`, `sameSite: 'lax'`, `path: '/api/auth'`, `maxAge` = the refresh lifetime in milliseconds. `res.clearCookie('rt', { path: '/api/auth' })` — the `path` must match or the browser keeps it.
`toFrontendRole(roleCodes)`	The rule from B4: includes `owner` or `admin` → `'admin'`; else the first of [teacher, accountant, librarian, receptionist, warden, transport_manager, nurse, security, parent, student] that is present; else `'receptionist'`. With no school (empty roles) return `null`.

`toFrontendUser(user, tenant | null)`	The shape your `auth.http.ts` reads: `{ id, name: user.fullName, email, role, permissions, avatar: user.avatarUrl, phone, organization }` where `permissions` is `['*']` if the set contains `*`, else the sorted array (empty if no school), and `organization` is `{ id, slug, name, roles }` or `null`.
`sendSession(res, status, { session, user, tenant, organizations })`	Sets the refresh cookie, then sends (using your `respond` helpers) `data: { accessToken, expiresIn: config.JWT_ACCESS_TTL, user: toFrontendUser(...), organizations }`. **The refresh token must never be in the JSON.**

> **Why the cookie path is /api/auth**
>
> The browser then sends `rt` only to `/api/auth/*` routes, not to every API call. Your Vite proxy and nginx keep the `/api` prefix, so the path matches in development and production.

> **Check**
>
> `npm run typecheck` passes (errors about files you haven't written yet are expected; fix everything else). You will exercise this code for real in C6.

C6. Sign up

Goal: `POST /api/auth/signup` creates an account and signs the person in. MUST

Request body (zod, `.strict()`): `{ "fullName": "Asha Kumar", "email": "asha@example.com", "password": "at-least-10-characters" }`.

Rules

- fullName 2–120 (trimmed); email valid, max 254, **stored lower-case**; password 10–128. Unknown fields → 400.
- Store only the **argon2 hash** (`await argon2.hash(password)`). Never return it: use `select` when creating the user.
- Email already used (any letter case) → **409** `{ "message": "An account with this email already exists" }`. Let the **unique constraint** detect it and catch Prisma error `P2002` (Part A4) — do not check first.
- Success → **201**. The person has no school yet, so the session has `tenantId = null`, `organization: null`, `organizations: []`, `role: null`. Use `createSession` + `sendSession`.
- Rate limit signup and login: `rateLimit({ windowMs: 60_000, limit: config.AUTH_RATE_LIMIT, ... })` as a middleware in the route: `router.post('/signup', limiter, handler)`.

Structure: `auth.schemas.ts` (zod schemas), `auth.service.ts` (`signup(input, meta)` — creates the user and returns what the route needs), `auth.routes.ts` (`publicAuthRouter`: `schema.parse(req.body)`, call the service, `sendSession`). Mount it in `app.ts`: `app.use('/api/auth', publicAuthRouter)` **above** the 404 handler (and, later, above `authenticate`). Client info for the session: `{ ip: req.ip, userAgent: req.get('user-agent') }`.

Check

terminal

```
API=http://localhost:5000/api
H='content-type: application/json'
curl -i -c jar.txt $API/auth/signup -H "$H" -d '{"fullName":"Asha
Kumar","email":"asha@example.com","password":"Temp-pass-12345"}'
#   201; body {"success":true,"data":{"accessToken":"...","expiresIn":900,"user":
{...,"role":null,"organization":null},"organizations":[]}}
#   headers: Set-Cookie: rt=...; Path=/api/auth; HttpOnly; SameSite=Lax    (NO refresh token in the body!)
curl -i $API/auth/signup -H "$H" -d '{"fullName":"Asha Again","email":"ASHA@example.com","password":"Temp-
pass-12345"}'   # 409
curl -i $API/auth/signup -H "$H" -d '{"fullName":"X","email":"nope","password":"short"}'
# 400 with "errors":[{field,message},...]
curl -i $API/auth/signup -H "$H" -d '{"fullName":"Asha","email":"b@example.com","password":"Temp-pass-
12345","isAdmin":true}'  # 400 (unknown field)
```

In Prisma Studio: the email is lower-case; `password_hash` starts with `$argon2id$`; `refresh_tokens` has one row whose `token_hash` is 64 hex characters (not the cookie value). Add `jar.txt` and `old-jar.txt` to `.gitignore` now — they hold cookies and must never be committed.

C7. Sign in

Goal: `POST /api/auth/login`. MUST

Request: `{ "email": "...", "password": "...", "organization": "greenfield" }` — `organization` (a slug) is **optional**; your frontend does not send it. Strict schema.

Rules

- Find the user by lower-cased email (`deletedAt: null`). Verify with `argon2.verify(hash, password)`.
- **Every failure gives the same answer: 401** `{ "message": "Invalid email or password" }` (this text is shown to users by your login form). Unknown email, wrong password, disabled account: identical.
- **Same timing for unknown emails:** if no user, still run `argon2.verify` against a dummy hash created once at module load (`const dummyHash = argon2.hash('x')` is a Promise; `await` it where used). Otherwise unknown emails answer faster and leak which emails exist.
- Only `status = 'active'` accounts sign in. On success set `last_login_at`.
- Choose the school: `pickTenantId(user.id, body.organization)`. Then `loadTenantContext` for it (or `null`). Create the session for that tenant, answer **200** with `sendSession` including `organizations: await listMyOrganizations(user.id)`.
- Same rate limiter as signup.

Check

terminal

```
curl -i -c jar.txt $API/auth/login -H "$H" -d '{"email":"asha@example.com","password":"Temp-pass-12345"}'
# 200 + Set-Cookie rt
curl -i $API/auth/login -H "$H" -d '{"email":"asha@example.com","password":"wrong-password-1"}'   # 401
Invalid email or password
curl -i $API/auth/login -H "$H" -d '{"email":"nobody@example.com","password":"wrong-password-1"}' # 401,
IDENTICAL body
```

Compare the two 401 responses: the same body, and roughly the same time (`curl -w "%{time_total}\n" -o /dev/null -s ...`).

C8. authenticate and /me

Goal: the middleware that turns a token into `req.user` and `req.tenant`, and the first protected route. MUST

Rules for `middleware/authenticate.ts` (export `AuthUser`, `currentUser(req)`, `currentTenant(req)`, `authenticate`):

- Header must be `Authorization: Bearer <token>`, else `HttpError(401, 'Unauthorized')`.
- `jwt.verify(token, secret, { algorithms: ['HS256'] })` throws on a bad signature or expiry: catch it, same generic 401. **Always pin the algorithm.** The payload type is `string | JwtPayload`: check it is an object with a string `sub`; `tid` is optional (string).
- **Load the person from the database on every request** (`id` = sub, `status: 'active'`, `deletedAt: null`; select `id, email, fullName, phone, avatarUrl`); not found → 401.
- If the token has `tid`: `loadTenantContext(user.id, BigInt(tid))`. **Null** (membership suspended, school suspended) → **401**, so the frontend tries a refresh, which falls back to another school or none (C9).
- Set `req.user` and (if any) `req.tenant`; `next()`.
- `currentUser(req)` returns `req.user` or throws 401. `currentTenant(req)` returns `req.tenant` or throws **403** `'Create or select a school first'`.

Route (a `privateAuthRouter` in `auth.routes.ts`): `GET /api/auth/me` → `ok(res, { ...toFrontendUser(user, tenant), organizations })` — the **user object itself** in `data` (your `getCurrentUser` reads it directly), plus the school list. In `app.ts`, add `app.use('/api', authenticate)` (the dividing line) and below it `app.use('/api/auth', privateAuthRouter)`.

Check

terminal

```
TOKEN=$(curl -s $API/auth/login -H "$H" -d '{"email":"asha@example.com","password":"Temp-pass-12345"}' |
node -pe "JSON.parse(require('fs').readFileSync(0,'utf8')).data.accessToken")
curl -i $API/auth/me                                  # 401 (no token)
curl -i $API/auth/me -H "Authorization: Bearer abc"     # 401 (garbage)
curl -s $API/auth/me -H "Authorization: Bearer $TOKEN"  # 200 {"success":true,"data":
{id,name,email,role:null,permissions:[],organization:null,organizations:[]}}
# Prisma Studio: set users.status='disabled' for Asha; call /me with the SAME token -> 401. Set it back to
'active'.
```

C9. Refresh and logout

Goal: keep people signed in, safely. This is the most delicate code in the project — go slowly. MUST

`POST /api/auth/refresh` (public, no body, uses the cookie `rt`; returns the same response as login):

1. Read `req.cookies.rt` (cookie-parser). Missing → 401.
2. Find the row by `tokenHash: sha256(cookie)`. Not found → clear the cookie, 401.
3. **Already used or revoked?** If `revokedAt` is set → 401. If `usedAt` is within the last **10 seconds** → **409** `{ "message": "Session refresh in progress" }` and **revoke nothing** (a second browser tab or a React double-start raced; your frontend retries after 300 ms and succeeds because the browser already holds the new cookie). If `usedAt` is older → theft signal: revoke the whole **family** (`updateMany({ where: { familyId, revokedAt: null }, data: { revokedAt: new Date(), revokedReason: 'reuse_detected' } })`), clear the cookie, 401.
4. Expired (`expiresAt < now`) → 401.
5. **Claim it atomically:** `updateMany({ where: { id, usedAt: null, revokedAt: null }, data: { usedAt: new Date() } })`; if `count !== 1` another request won the race → **409**.

6. Load the user (active, not deleted) — else 401. Decide the school: if `row.tenantId` is set and `loadTenantContext(user.id, row.tenantId)` is not null, keep it; otherwise `pickTenantId(user.id)` (a fallback school, or none).
7. `createSession({ userId, tenantId, meta, familyId: row.familyId })` → `sendSession` (200).

`POST /api/auth/logout` (public): if the cookie exists and matches a row, revoke its family (`revokedReason: 'logout'`). Always `clearRefreshCookie` and answer **204** — even with no cookie.

Why a family? One sign-in creates a chain of refresh tokens (each refresh issues the next). Revoking the whole chain signs that device out everywhere, which is what you want if a copy leaks.

Check

terminal

```
curl -s -c jar.txt $API/auth/login -H "$H" -d '{"email":"asha@example.com","password":"Temp-pass-12345"}' >
/dev/null
cp jar.txt old-jar.txt                                  # keep a copy of the FIRST cookie
curl -i -b jar.txt -c jar.txt -X POST $API/auth/refresh   # 200, new accessToken, NEW cookie value in
jar.txt
curl -i -b old-jar.txt -X POST $API/auth/refresh          # immediately replaying the OLD cookie: 409
(within 10 s)
sleep 12
curl -i -b old-jar.txt -X POST $API/auth/refresh          # now 401, and the whole family is revoked:
curl -i -b jar.txt -X POST $API/auth/refresh              # the NEW cookie is dead too -> 401
curl -i -X POST $API/auth/refresh                         # no cookie -> 401
# logout:
curl -s -c jar.txt $API/auth/login -H "$H" -d '{"email":"asha@example.com","password":"Temp-pass-12345"}' >
/dev/null
curl -i -b jar.txt -c jar.txt -X POST $API/auth/logout    # 204 and Set-Cookie clearing rt
curl -i -b jar.txt -X POST $API/auth/refresh              # 401
```

In `refresh_tokens` (Studio) watch `used_at` , `revoked_at` and `revoked_reason` change as you run this.

C10. Schools: create, list, switch, current

Goal: a signed-in person creates a school and becomes its owner; people with several schools can switch. MUST

10a. The permission catalogue and the seed

Create `src/permissions.ts` with:

- `ACTIONS` = `['view','create','edit','delete','import','export','approve','print','download','manage']` and `MODULES` = the 11 modules of B4.
- `PERMISSIONS`: **generated** with `MODULES.flatMap(...)` and `ACTIONS.map(...)` → `{ code: module + '.' + action, module, action, description }` (110 entries).
- `DEFAULT_ROLES`: array of `{ code, name, description, permissions }` with the 12 roles of B4. `owner` has an empty list (wildcard in code); `admin` = every code except `organization.manage` (`PERMISSIONS.map(p => p.code).filter(...)`); the others as in the B4 table. *Hint:* a helper `const all = (m: string) => ACTIONS.map(a => m + '.' + a)` keeps this short.

`prisma/seed.ts`: for every entry of `PERMISSIONS` run `prisma.permission.upsert({ where: { code }, update: {...}, create: {...} })`, print how many, and disconnect. Safe to run repeatedly. Run `npx prisma db seed` now.

> **Check**
> Prisma Studio → `permissions` has **110** rows (e.g. `students.create`).

10b. `POST /api/organizations` — create a school

Mounted after `authenticate`. Body (strict): `{ "name": "Greenfield School", "slug": "greenfield", "timezone": "Asia/Kolkata" }`.

- name 2–120 (trimmed). slug: regex `/^[a-z0-9][a-z0-9-]{2,40}$/` and **not reserved** (`api, admin, app, www, platform, karkaii, support, login, signup`; use zod `.refine()`). timezone optional (default "UTC").
- Duplicate slug → **409** (catch `P2002`). Success → **201** `{ id, slug, name, timezone, roles: ['owner'] }`.
- Rate limit it (20 per hour per IP).

Do all of this in **one** `prisma.$transaction(async (tx) => { ... })`:

1. Create the organization.
2. Load all permissions (`tx.permission.findMany()`) into a map code → id. If a code in `DEFAULT_ROLES` is missing, **throw an Error** saying "run: npx prisma db seed".
3. For each entry of `DEFAULT_ROLES`: create the role (`tenantId` = the new id, `isSystem: true`), then `createMany` its `role_permissions` (none for owner). Remember the owner role id.
4. Create the creator's **membership** with the owner role in one call: `tx.membership.create({ data: { tenantId, userId, roles: { create: [{ roleId, tenantId }] } } })`.

List mine: `GET /api/organizations` → `ok(res, await listMyOrganizations(user.id))`.

10c. `POST /api/auth/switch-organization`

Body (strict): `{ "organization": "sunrise" }` (a slug). Authenticated (put it in `privateAuthRouter`).

- Load the person's membership for that slug (active, school active). None → **403** `'You do not have access to this organization'` — the **same answer** whether the school does not exist or the person is not in it, so nobody can probe which schools exist.
- Take over the current login session: read `req.cookies.rt`; if it matches an **unused, unrevoked** row of **this same user**, claim it (same atomic `updateMany` as C9) and reuse its `familyId`; otherwise start a new family.

- `createSession({ userId, tenantId: newSchoolId, meta, familyId })` → `sendSession` (200), with `organizations` included. The frontend swaps in the new token.

10d. `requirePermission` and the current school

`middleware/requirePermission.ts` : a function that **returns** a middleware: `requirePermission('organization.edit')` (accepts several; all required). It uses `currentTenant(req)` (403 if none) and `hasPermission` ; missing → **403** `'You do not have permission to do this'` .

Route	Behaviour
`GET /api/organizations/current` requirePermission('organization.view')	`{ id, slug, name, timezone, status, createdAt, myRoles, myPermissions }` for the school in the token.
`PATCH /api/organizations/current` requirePermission('organization.edit')	Strict body, both optional: `name` , `timezone` . `updateMany({ where: { id: tenant.id, deletedAt: null }, data })` ; check `count` . Returns the same as GET. (A field set to `undefined` is left unchanged.)

> **Where the tenant id comes from**
>
> In the PATCH query the `where` uses `req.tenant.id` — the id of a school the caller was **verified** to belong to this very request — never a value from the body or URL. Repeat that habit in every future module and isolation takes care of itself.

Check

terminal

```
# sign in as Asha (no school yet), then create one
TOKEN=$(curl -s $API/auth/login -H "$H" -d '{"email":"asha@example.com","password":"Temp-pass-12345"}' |
node -pe "JSON.parse(require('fs').readFileSync(0,'utf8')).data.accessToken")
curl -i $API/organizations -H "$H" -d '{"name":"Greenfield School","slug":"greenfield"}'
# 401
curl -i $API/organizations -H "$H" -H "Authorization: Bearer $TOKEN" -d '{"name":"Greenfield
School","slug":"greenfield"}'   # 201 roles ["owner"]
curl -i $API/organizations -H "$H" -H "Authorization: Bearer $TOKEN" -d
'{"name":"Copy","slug":"greenfield"}'         # 409
curl -i $API/organizations -H "$H" -H "Authorization: Bearer $TOKEN" -d '{"name":"Admin","slug":"admin"}'
# 400
# switch into it: you get a NEW token that carries the school
curl -s -c jar.txt $API/auth/switch-organization -H "$H" -H "Authorization: Bearer $TOKEN" -d
'{"organization":"greenfield"}'
TOKEN=...paste the new accessToken...
curl -s $API/organizations/current -H "Authorization: Bearer $TOKEN"       # school + myRoles ["owner"],
myPermissions ["*"]
curl -s -X PATCH $API/organizations/current -H "$H" -H "Authorization: Bearer $TOKEN" -d
'{"name":"Greenfield International"}'   # 200
curl -i $API/organizations/current -H "Authorization: Bearer <the OLD token without a school>"   # 403
"Create or select a school first"
curl -i $API/auth/switch-organization -H "$H" -H "Authorization: Bearer $TOKEN" -d
'{"organization":"nowhere"}'       # 403
```

In Studio you should see: 1 organization, **12 roles**, `role_permissions` rows for the non-owner roles, 1 membership, 1 membership_role (owner). To prove the transaction works, temporarily misspell a permission code in `DEFAULT_ROLES` : creating a school must fail with a 500 and leave **no organization row behind**. Fix it back.

C11. Roles, permissions and members

Goal: a school owner can see the permission catalogue and roles, add people to the school, change their roles and switch them off. MUST members + read-only roles · SHOULD role create/edit/delete.

All routes here are mounted after `authenticate`, work inside the school in the token, and use `requirePermission`. Every query uses `tenantId: currentTenant(req).id`. Ids in URLs go through `parseId`.

11a. Read-only: permissions and roles

Route	Behaviour
`GET /api/permissions` roles.view	The whole catalogue, flat: `[{ code, module, action }]` sorted by module then action. (The frontend groups it into its matrix.)
`GET /api/roles` roles.view	The school's roles: `[{ id, code, name, description, isSystem, memberCount, permissions }]` where `permissions` is the list of codes (for `owner` return `['*']`). *Hint:* `_count` in Prisma `select` gives the number of related membership rows.

11b. Members

A **member** is a membership row. Its id (`:id` in URLs) is the **membership id**. Member JSON: `{ id, userId, name, email, phone, status, roles: ['teacher'], createdAt }`.

Route	Rules
`GET /api/members?page=&limit=&q=` members.view	Members of this school. `limit` default 20, max 100; `q` searches name and email (case-insensitive `contains`); ordered by membership id. Response: `ok(res, rows, { page, limit, total, pages })` (the **meta** your frontend's `toPaginated` reads). Use `prisma.$transaction([findMany, count])`. Query values are text: `z.coerce.number()`.
`POST /api/members` members.create	Body strict: `{ email, fullName?, password?, roles: string[] (1–5 role codes) }`. Roles are looked up **inside this school** (unknown → 400 `Unknown role: x`). Assigning `owner` needs the caller to be an owner (else 403). Then: if an account with that email **exists** → just create the membership (already a member of this school → **409**; `fullName` / `password` are ignored). If it **does not exist** → `fullName` and `password` (10+) are required (else 400 with `errors`), create user + membership together in a **transaction**. 201 + the member.
`PATCH /api/members/:id` members.edit	Body strict, at least one of: `status` (`active` | `suspended`), `roles` (replaces the whole set; `deleteMany` + `createMany` in a transaction). Look the member up with `where: { id, tenantId }` (404 if absent). Guards: you cannot change **your own** membership (400); changing an owner, or granting owner, needs the caller to be an owner (403); **a school must always keep at least one active owner** (count active owner memberships after the change; fewer than 1 → 400).
`DELETE /api/members/:id` members.delete	Same guards as PATCH. Deletes the membership (its role links cascade). The person's account stays (they may belong to other schools). 204. SHOULD

> **Hint: "is the caller an owner?"**
> `currentTenant(req).roles.includes('owner')`. And "is the target an owner?" — select the target's role codes together with the membership.

11c. Role management SHOULD

Route	Rules
`POST /api/roles` roles.create	Body strict: `{ code, name, description?, permissions: string[] }`; `code` matches `/^[a-z][a-z0-9_]{2,40}$/`. Every permission code must exist (else 400 listing the unknown ones). Duplicate code in this school → 409. Creates a **custom** role (`isSystem: false`) and its `role_permissions` in one transaction. The code `owner` is reserved.
`PATCH /api/roles/:id` roles.edit	**Custom roles only** (system role → 403 `System roles cannot be changed`). Optional `name` , `description` , `permissions` (replaces the set).
`DELETE /api/roles/:id` roles.delete	Custom only. If any membership uses it → 409 `Role is in use` . 204.

Check (as the owner of greenfield, token carrying the school)

terminal

```
AUTH="Authorization: Bearer $TOKEN"
curl -s $API/permissions -H "$AUTH" | head -c 400                          # catalogue
curl -s $API/roles -H "$AUTH"                                               # 12 roles; owner permissions
["*"]
curl -i $API/members -H "$H" -H "$AUTH" -d '{"email":"ravi@example.com","fullName":"Ravi
Kumar","password":"Temp-pass-12345","roles":["accountant"]}'   # 201 (new account + membership)
curl -s "$API/members?limit=10&q=ravi" -H "$AUTH"                            # list with meta
{page,limit,total,pages}
curl -i $API/members -H "$H" -H "$AUTH" -d '{"email":"ravi@example.com","roles":["teacher"]}'
# 409 already a member
curl -i $API/members -H "$H" -H "$AUTH" -d '{"email":"x@example.com","roles":["teacher"]}'
# 400 (new account needs fullName + password)
curl -i $API/members -H "$H" -H "$AUTH" -d '{"email":"y@example.com","fullName":"Y","password":"Temp-pass-
12345","roles":["wizard"]}'   # 400 Unknown role
MID=<the id of Ravi's membership>
curl -s -X PATCH $API/members/$MID -H "$H" -H "$AUTH" -d '{"roles":["teacher"]}'
# 200 roles ["teacher"]
curl -i -X PATCH $API/members/<Asha's own membership id> -H "$H" -H "$AUTH" -d '{"status":"suspended"}'
# 400 cannot change your own access
```

The multi-school proof: sign in as Ravi (`ravi@example.com`) — he lands in Greenfield as `accountant` / `teacher` with only that role's permissions (`GET /auth/me` shows `role` and `permissions` ; `PATCH /organizations/current` → 403; `GET /members` → 403). Then have Ravi create his own school and `switch-organization` into it: there he is `owner` with `["*"]` . One account, two schools, two different powers. Finally, as Asha, suspend Ravi's membership and call `/auth/me` with Ravi's old token: 401 immediately.

C12. Connect your React frontend

Goal: in your real app you can sign up, create a school, sign in and out, stay signed in across reloads and 15-minute token expiry, and the UI reacts to permissions. MUST (3.5 hours)

All paths are in `~/karkaii/karkaii-frontend/src`. The frontend already has the machinery (API client with automatic refresh, an `AuthProvider`, an HTTP auth service written for the old server). Most of this chapter is *adapting* that, not building from scratch. Read each file before editing it.

> **Work on a branch, never on main**
>
> `cd ~/karkaii/karkaii-frontend && git checkout -b feat/real-auth`. `main` is protected and every merge deploys the live site, so open a pull request only when this chapter's checks pass.

F1. Turn real auth on — and only auth

The mock/real switch is **one global flag** (`VITE_USE_MOCK_API`): setting it to `false` would send students and dashboard calls to endpoints that do not exist yet. So give auth its own flag.

1. Create `.env.development` (git-ignored) in the frontend root:

   .env.development

   ```
   VITE_APP_ENV=development
   VITE_USE_MOCK_API=true        # students + dashboard stay on mock data
   VITE_REAL_AUTH=true           # NEW: auth talks to your backend
   VITE_AUTH_ENABLED=true        # routes now require sign-in
   VITE_API_BASE_URL=/api
   API_PROXY_TARGET=http://localhost:5000
   ```

2. `config/env.ts`: add `useRealAuth: boolean` to `AppConfig` and set it with `toBoolean(raw.VITE_REAL_AUTH, false)` (copy how `authEnabled` is done). If TypeScript complains about the env name, declare it in `vite-env.d.ts` like the others.
3. `services/auth/auth.service.ts`: replace `createService(authMockService, authHttpService)` with `env.useRealAuth ? authHttpService : authMockService` (import `env` from `@config/env`).

> **Check**
>
> Start the backend (`npm run dev` in `backend`) and the frontend (`npm run dev`). Open `/login`, sign in as Asha (from C6/C7). In the browser's DevTools → Network you see `POST /api/auth/login` answer 200 (not a mock). If you land on a dashboard, auth works (it still shows mock data, which is expected). Application → Cookies: `rt` is there, **HttpOnly**, path `/api/auth`.

F2. Map the backend's user to the frontend's user

1. `services/auth/auth.types.ts`: add `interface OrganizationRef { id: string; slug: string; name: string; roles: string[] }`; extend `AuthUser` with `organization: OrganizationRef | null` and `organizations: OrganizationRef[]`. Add `SignupPayload { fullName; email; password }` and `CreateOrganizationPayload { name; slug; timezone? }`.
2. `services/auth/auth.http.ts`:
   - `BackendUser`: add `permissions: string[]`, `organization: OrganizationRef | null`; `role` may be `null`.
   - `toAuthUser`: use `u.permissions` instead of the old "admin gets ['*']" rule (delete `FULL_ACCESS_ROLES`); `role: u.role ?? 'receptionist'` (a neutral role for people with no school yet; the guard in F5 redirects them); carry `organization` and `organizations`.

- `LoginResponse` now also has `expiresIn` and `organizations` at the top level: merge them into the user inside `toSession`.
- The `getCurrentUser` response is the user object itself with `organizations` inside it (C8).

3. `services/api/endpoints.ts`: add `auth.signup: '/auth/signup'`, `auth.switchOrganization: '/auth/switch-organization'`, and `organizations: { list: '/organizations', create: '/organizations' }`.
4. `auth.service.ts` (the `AuthService` interface), `auth.http.ts` and `auth.mock.ts`: add three methods — `signup(payload)` (POST, `skipAuth: true, skipRefresh: true`, returns an `AuthSession` via `toSession`), `createOrganization(payload)` (POST, authenticated, returns the organization), `switchOrganization(slug)` (POST `{ organization: slug }`, returns an `AuthSession`). The mock versions can throw "not available in mock mode"; the interface forces both files to stay in sync.
5. `app/providers/AuthProvider.tsx` and `auth-context.ts`: add `applySession(session: AuthSession)` — it does what the end of `login` does: `tokenStorage.setTokens(session.accessToken, undefined, tokenStorage.isPersisted())`, `tokenStorage.setUser(session.user, ...)`, `setUser(session.user)`, `setStatus('authenticated')` — and expose it in the context value.

> **Check**
> `npm run build` (or `npx tsc -b`) in the frontend has no new TypeScript errors. Sign in again: `useAuth().user.permissions` is now what the backend sent (log it once to see it).

F3. Sign-up page

Your register screens are template markup. The routed one is `views/auth/register/register.tsx` (path `/register` in `routes/all_routes.tsx`).

- Study how `views/auth/login/login.tsx` uses `features/auth/hooks/useLogin.ts` (state, submit, error display) and copy that pattern into a `useSignup` hook in the same folder.
- Fields: full name, email, password (+ confirm, checked on the client only). On submit: `await authService.signup(...)`; then `await login({ email, password })` from `useAuth()` (two calls, but zero provider changes); then navigate to the create-school page (F4).
- Show server errors: a 409 message under the email; for 400 validation use `apiError.fieldErrors` (`{ fullName?, email?, password? }`) to mark the right inputs. Disable the button while submitting.

> **Check**
> Register a new person in the browser: Network shows `POST /api/auth/signup` 201 then `POST /api/auth/login` 200. Try an existing email: the page shows the 409 message. Try a short password: the password field shows the validation message.

F4. Create-school page

- Add a route constant `createSchool: '/create-school'` to `routes/all_routes.tsx` and a lazy route in `routes/route_config.tsx` (copy how `Register` is declared there). New view: `views/auth/createSchool/createSchool.tsx`, styled like the register page (reuse its layout classes).
- Form: school name; slug (auto-filled from the name — lower-case, spaces to hyphens, strip other characters — but editable); timezone (a select with `Asia/Kolkata` as default).
- Submit: `await authService.createOrganization(payload)` → `const session = await authService.switchOrganization(payload.slug)` → `applySession(session)` → `navigate(roleHomeRoute(session.user.role))`. Show 409 ("slug taken") and validation errors on the fields.

> **Check**
> After creating a school you land on the admin dashboard; reloading the page keeps you signed in (the app restores the session from the cookie); `useAuth().user.organization.slug` is your slug and `permissions` is `['*']`.

F5. Route people to the right place

- After **login** (in `useLogin.ts`): if `user.organization === null` go to `all_routes.createSchool`, else `roleHomeRoute(user.role)`.
- **The guard:** find the component that protects routes (look in `components/routing/` and at how `route_config.tsx` wraps pages). If the user is authenticated but has no organization and is not already on the create-school page, redirect them there. Make the create-school route itself require a signed-in user but *not* an organization.

> **Check**
> A person who signed up but has not created a school can only reach `/create-school`; anything else redirects. A person with a school never sees that page again unless they open the URL.

F6. Session behaviour (mostly free; just verify)

Your `apiClient` already retries a 401 once after calling `POST /auth/refresh`, and `AuthProvider` already retries a 409. To *see* it work, set `JWT_ACCESS_TTL=60` in the backend `.env` and restart it. Sign in, then use the app for a couple of minutes: in Network you will see a request fail with 401, then `POST /api/auth/refresh` 200, then the same request succeed — with no visible interruption. Restore 900 afterwards. Also test: sign out (`POST /api/auth/logout` 204, cookie cleared); open two tabs and refresh both at the same moment (a 409 may appear, then success — that is the 10-second grace working).

F7. Use permissions in the UI SHOULD

`RoleGuard` (`components/routing/RoleGuard.tsx`) and `useAuth().hasPermission('students.create')` already exist and understand `'*'`. Pick two or three places and use them: hide an "Add student" button unless `hasPermission('students.create')`; hide a sidebar entry unless `hasPermission('finance.view')`. Test with Ravi (accountant: he sees finance, not admissions) and Asha (owner: sees everything).

F8. School switcher SHOULD

People with several schools need a way to move between them. In the header (find the user dropdown in `layouts/`), list `user.organizations` with the current one ticked. Clicking another calls `authService.switchOrganization(slug)`, then `applySession(session)`, then navigates to `roleHomeRoute(session.user.role)`. **Check:** as Ravi (member of Greenfield, owner of his own school) switch schools: the role, menu and permissions change immediately.

F9. Roles, permissions and members screens COULD

`views/userManagement/roles-permissions` and `permission.tsx` are static template data today (`rolesPermissions.data.ts`). If you have spare hours, add a `rolesService` (copy the structure of `services/students/`: types, mock, http, service) calling `GET /roles`, `GET /permissions`, `GET /members`, and bind the lists. The permission matrix's ten switches per module map to `module.view` … `module.manage`. Do this last; the demo does not depend on it.

C13. Deploy to the droplet with the DigitalOcean database

Goal: your live site (`http://64.227.129.36/app/` today) signs people up and in against the DigitalOcean database. SHOULD (2 hours). **Do not start before C0–C11 pass their checks locally.**

> **Cookies and HTTPS**
> Browsers silently drop `Secure` cookies on plain `http://`. If the site is still served at a bare IP without HTTPS, sign-in would appear to work and then log you out. Until you have a domain and a certificate, set `COOKIE_SECURE=false` in the server env file, and plan to get HTTPS (certbot) as soon as possible: tokens and passwords sent over plain HTTP can be read by anyone on the network. With HTTPS use `COOKIE_SECURE=true`.

D1. Prepare the database from your laptop (against DigitalOcean)

Your laptop's IP must be in the cluster's trusted sources (C0.2). This applies the migrations you created locally and loads the permission catalogue:

terminal (in ~/karkaii/backend)

```
set -a; source .env.do; set +a        # loads DATABASE_URL for DigitalOcean into this shell only
npx prisma migrate status                  # connects; lists the migration(s) not yet applied
npx prisma migrate deploy                  # creates the tables on DigitalOcean (this is the PostgreSQL 18 test)
npx prisma db seed                         # the 110 permissions
exit                                       # (or open a new shell) so DATABASE_URL does not linger
```

> **Check**
> No errors, and `docker run --rm postgres:16 psql "YOUR_DO_URL_WITHOUT_connection_limit" -c "\dt"` lists your 9 tables (+ `_prisma_migrations`). If you see a certificate error, apply the TLS note from C0.4.

D2. Prepare the droplet (once)

terminal: ssh root@64.227.129.36

```
# Node 22
curl -fsSL https://deb.nodesource.com/setup_22.x | bash -
apt-get install -y nodejs

# a locked-down service user and folders
useradd --system --home /opt/karkaii --shell /usr/sbin/nologin karkaii
mkdir -p /opt/karkaii/api /etc/karkaii
chown -R karkaii:karkaii /opt/karkaii/api
```

Create the environment file `/etc/karkaii/api.env` (`nano`), then lock it down: `chown root:karkaii /etc/karkaii/api.env && chmod 640 /etc/karkaii/api.env`.

/etc/karkaii/api.env

```
NODE_ENV=production
PORT=5000
DATABASE_URL="postgresql://karkaii_app:PASSWORD@HOST:25060/karkaii?sslmode=require&connection_limit=5"
JWT_ACCESS_SECRET=generate-a-NEW-one-here-with-openssl-rand-hex-32
JWT_ACCESS_TTL=900
REFRESH_TTL_DAYS=30
AUTH_RATE_LIMIT=10
COOKIE_SECURE=false
```

(Use a **different** `JWT_ACCESS_SECRET` from your laptop's. Set `COOKIE_SECURE=true` as soon as HTTPS works.) If the cluster shows a private (VPC) hostname and the droplet is in the same region, use that host instead.

The service definition `/etc/systemd/system/karkaii-api.service`:

/etc/systemd/system/karkaii-api.service

```
[Unit]
Description=Karkaii API
After=network.target

[Service]
User=karkaii
WorkingDirectory=/opt/karkaii/api
EnvironmentFile=/etc/karkaii/api.env
ExecStart=/usr/bin/node dist/server.js
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

D3. Release (repeat for every deployment)

First move `prisma` from `devDependencies` to `dependencies` in `package.json` (so Prisma's client generation works after `--omit=dev`), then:

terminal (laptop, in ~/karkaii/backend)

```
npm run build
rsync -az --delete dist package.json package-lock.json prisma root@64.227.129.36:/opt/karkaii/api/
ssh root@64.227.129.36 "cd /opt/karkaii/api && npm ci --omit=dev && chown -R karkaii:karkaii /opt/karkaii/api"
ssh root@64.227.129.36 "systemctl daemon-reload && systemctl enable --now karkaii-api && sleep 2 && systemctl
status karkaii-api --no-pager | head -12"
ssh root@64.227.129.36 "curl -s http://127.0.0.1:5000/api/health"      # {"success":true,"data":{"status":"ok"}}
# if anything fails:  ssh root@64.227.129.36 "journalctl -u karkaii-api -n 50 --no-pager"
```

The service uses the DigitalOcean database; `migrate deploy` and the seed already ran from your laptop in D1 (run them again whenever you add a migration).

D4. nginx: send `/api/` to the API

Find the site configuration: `grep -rl karkaii /etc/nginx/sites-enabled/` (it is the file that serves the landing page and `/app/`). **Back it up** (`cp file file.bak`), then inside the `server { ... }` block add:

nginx: inside the server block

```
location /api/ {
    proxy_pass http://127.0.0.1:5000;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

terminal (droplet)

```
nginx -t && systemctl reload nginx          # only reload if the test says "ok"
curl -s http://64.227.129.36/api/health      # from your laptop: {"success":true,"data":{"status":"ok"}}
```

If `nginx -t` fails, restore the backup. If the configuration looks different from what you expected, send it to me before changing it — this server also serves your live site.

D5. Frontend build flags

The frontend workflow bakes the flags in at build time. In the **frontend** repo, on your branch, edit `.github/workflows/deploy.yml` — in the build step's `env:` keep `VITE_USE_MOCK_API: 'true'`, change `VITE_AUTH_ENABLED: 'true'`, and add `VITE_REAL_AUTH: 'true'` and `VITE_API_BASE_URL: '/api'` (keep `VITE_BASE_PATH=/app/`). Open a pull request; **merging it deploys the live site**, so merge only after the whole acceptance checklist passes locally. After the deploy: open `http://64.227.129.36/app/`, sign up, create a school, reload, sign out and in again.

> **Check**
> On the live site DevTools → Network shows `/api/auth/...` calls with 200/201 and the `rt` cookie is set. In DigitalOcean → Databases → your cluster → Insights (or Prisma Studio via `.env.do`) you can see the new user. If sign-in succeeds but you are logged out on the next page, it is the **Secure cookie** problem above.

C14. Acceptance checklist

Tick every row (curl commands are in the steps; the UI rows are in the browser). Done when all MUST rows pass.

✓	Behaviour	Expected
	API — `GET /api/health`	200 envelope; with the DB stopped a clean 500 JSON
	Sign up with valid data	201; cookie `rt`; **no** refresh token or hash in the body
	Same email in other letter case; short password; bad email; extra field	409; 400 with `errors[]`; 400; 400
	Sign in right / wrong password / unknown email	200 / 401 / 401 with the **same** message
	Rate limit: 11 sign-ins in a minute	429
	`GET /auth/me`: no token / garbage / valid	401 / 401 / 200 user object
	Disable the account in Studio, reuse the token	401 immediately
	Refresh with cookie; replay the old cookie at once; replay it after 10 s	200; 409; 401 + the whole family revoked
	Logout	204; cookie cleared; refresh then 401
	Create a school (signed in) / without token / duplicate slug / reserved slug	201 roles [owner] / 401 / 409 / 400
	A failed school creation leaves no partial rows	no orphan organization or roles
	New school has 12 roles; creator is owner	Studio check
	Switch school (member / not a member / nonexistent)	200 new token / 403 / 403 (identical bodies)
	School route with a token that has no school	403 "Create or select a school first"
	One person, two schools: different `role` and `permissions` per school	owner `['*']` vs a limited role
	Add a member (new account / existing account / already a member / unknown role)	201 / 201 / 409 / 400
	Limited member calls `PATCH /organizations/current` and `GET /members`	403
	Change a member's roles; try to change your own; remove the last owner	200; 400; 400
	Suspend a member, reuse their token	401 immediately
	School A cannot see school B's members (`GET /members`, `PATCH /members/:idOfB`)	only own members; 404
	UI — sign up in the browser; error messages appear on the right fields	works
	Create a school in the browser; land on the dashboard; reload stays signed in	works
	Token expiry handled silently (TTL 60 s test)	401 then refresh 200 then success

	Permission-based UI shows / hides for owner vs limited role SHOULD	works
	School switcher SHOULD	works
	Live — the same flows on the droplet SHOULD	works over the DigitalOcean DB

C15. If you have time left

1. **Lockout and audit log:** count wrong passwords (`failed_login_count`); at 5 set `locked_until` (15 minutes) and refuse even the right password until then; write rows to `audit_log` for signup, login, failed login, school created, member added/removed.
2. **Automated tests** with `vitest` + `supertest`, one per checklist row. The most valuable: "school B cannot touch school A".
3. **Forgot password** (your frontend already has the screens and endpoints `/auth/forgot-password`, `/auth/reset-password`): needs an email provider and a hashed, expiring, single-use reset token.
4. **Email verification** before a new account may create schools (otherwise anyone can sign up with someone else's email).
5. **HTTPS** with a domain and certbot, then `COOKIE_SECURE=true`.
6. **Re-sync default roles** when you add permissions (update every school's system roles from `DEFAULT_ROLES`).
7. **PostgreSQL Row-Level Security** as a second lock behind your `tenantId` filters.

Part D — Reference

D1. Troubleshooting

Symptom	Cause and fix
`Invalid environment (.env): ...` (your own check)	Fix the variable it names. The secret needs 32+ characters (`openssl rand -hex 32`).
`Environment variable not found: DATABASE_URL`	`.env` is missing or not in the project root.
`P1001: Can't reach database server` (local)	Docker container not running: `docker start karkaii-pg` .
`P1001` or a hang/timeout against DigitalOcean	Your IP (or the droplet) is not in **Trusted sources**; or wrong host/port (25060); or the password has special characters — URL-encode them (`@` → `%40` , `/` → `%2F`).
`P1011` / "certificate" / "self-signed" error (DigitalOcean)	Add `&sslaccept=accept_invalid_certs` to the URL, or download the CA certificate and add `&sslcert=/path/ca-certificate.crt` (C0.4).
`permission denied for schema public` / `permission denied to create ...` (DigitalOcean)	Run the two `GRANT` lines from C0.3 as `doadmin` , connected to the `karkaii` database.
`P3014` / shadow database error from `migrate dev`	You ran `migrate dev` against a limited user. Use `migrate dev` **only on the local database** and `migrate deploy` for DigitalOcean.
Prisma complains about PostgreSQL version 18	Send me the exact message; as a workaround test against a PostgreSQL 16/17 database.
`Cannot find module '.prisma/client'` / `PrismaClient is not a constructor`	`npx prisma generate` . On the droplet: `prisma` must be in `dependencies` before `npm ci --omit=dev` .
Prisma: missing opposite relation field	Every relation needs both sides; the message names the field to add.
`Do not know how to serialize a BigInt`	`import './lib/bigint'` must run (in `app.ts`).
TypeScript: `Property 'user' does not exist on type 'Request'`	`types/express.d.ts` missing, or `tsconfig.json` does not include `src` .
TypeScript: `'string' is not assignable to 'bigint'`	Ids from URLs or tokens are text: `BigInt(value)` after validating (use `parseId`).
400 `Validation failed` with "Required" / "Expected object"	curl needs `-H 'content-type: application/json'` with `-d` ; and `express.json()` must be registered before the routes.
400 `Unrecognized key(s)`	The schema is `.strict()` ; remove the extra field (or add it to the schema if the frontend really sends it).
`req.cookies` is undefined	`app.use(cookieParser())` missing or placed after the routes.
Cookie not set / not sent in the browser	Check DevTools → Application → Cookies. `Secure` cookie on plain http (set `COOKIE_SECURE=false`); the path is `/api/auth` (only auth calls carry it); in dev, call the API **through the Vite proxy** (same origin), not directly on port 5000.

Browser shows a CORS error	You are calling `localhost:5000` from a page on `localhost:5173` directly. Use the relative `/api` base so the Vite proxy handles it.
Signed out on every page load (frontend)	The refresh call fails: look at `POST /api/auth/refresh` in Network. Missing cookie, 401 (token reused after 10 s), or the cookie cleared by a path mismatch in `clearCookie`.
Login works but the dashboard says "session expired"	The frontend got 401 from another call: the backend `authenticate` rejected the token (wrong secret between processes, expired, or user/school suspended).
After merging the frontend PR, the live app is mock-only	The workflow build env still has the old flags (D5), or the browser cached the old build: hard refresh.
Every protected request returns 401	Header must be exactly `Authorization: Bearer <token>`. Token expired (15 min) — sign in again; user or school not `active`.
Sign-in 401 for the right password	Email stored lower-case? Lower-case it on save *and* on lookup. Is the stored hash a real argon2 hash?
403 "Create or select a school first"	The token has no `tid`: call `switch-organization` (or sign in again after creating a school).
Creating a school fails with "Permission … not in the database"	Run `npx prisma db seed` (on DigitalOcean too, D1). Schools created before seeding have empty roles: delete them and recreate.
429 Too Many Requests	Rate limit; wait a minute (or raise `AUTH_RATE_LIMIT` for local testing).
`Cannot set property query of # <IncomingMessage>`	Express 5 made `req.query` read-only: do not assign to it; pass it to `schema.parse(req.query)`.
A `dotenv` banner on every start	You have dotenv 17+; use `npm i dotenv@^16`.
`EADDRINUSE :::5000`	Another process (the old `server/`?) uses the port.
argon2 fails to install	Use Node 22 LTS; fallback `sudo pacman -S base-devel python`.
`node dist/server.js` not found	`tsconfig.build.json` needs `"rootDir": "src"` and `"include": ["src"]`.
An error disappears and the request hangs	An unawaited promise. Every Promise needs `await` (Part A1).
systemd: service fails to start (droplet)	`journalctl -u karkaii-api -n 50`. Usual causes: wrong path in `ExecStart`, missing `dist/`, a bad line in `api.env`, the DB not reachable (trusted sources).

Habits that save hours

(1) Read the *whole* error and the first stack line that is in *your* file. (2) `curl -i` shows status and headers; the browser Network tab shows what the frontend really sent. (3) Look at data in Prisma Studio instead of guessing. (4) Change one thing at a time and commit when something works.

D2. API summary

All routes are under `/api`. Success bodies are `{ success: true, data, meta? }`. "Person" routes need a Bearer token; "School" routes need a token that carries a school and the stated permission.

Method	Path	Needs	Does
GET	/health	public	API and database are up

POST	/auth/signup	public	Create account, sign in (no school yet). 201
POST	/auth/login	public	Token + user + my schools; cookie `rt`
POST	/auth/refresh	public (cookie)	New token + cookie. 409 = retry
POST	/auth/logout	public (cookie)	Revoke session, clear cookie. 204
GET	/auth/me	person	User (role, permissions, active school) + my schools
POST	/auth/switch-organization	person	Act in another of my schools; new token
POST	/organizations	person	Create a school; I become owner. 201
GET	/organizations	person	My schools with my roles
GET / PATCH	/organizations/current	organization.view / .edit	The active school; change name / timezone
GET	/permissions	roles.view	The permission catalogue (110)
GET	/roles	roles.view	Roles of the school with permissions and member counts
POST / PATCH / DELETE	/roles, /roles/:id	roles.create / edit / delete	Custom roles (SHOULD)
GET	/members	members.view	People in the school (page, limit, q)
POST	/members	members.create	Add a person (new or existing account). 201
PATCH / DELETE	/members/:id	members.edit / delete	Change roles or status / remove

D3. Security notes

Risk	How this design handles it
School A sees school B's data	The school comes from a membership verified on every request; every query uses that `tenantId`; identical 403 for "unknown" and "not yours".
XSS steals the refresh token	It lives in an **httpOnly** cookie, path-limited to `/api/auth`; only a hash is stored.
Stolen access token	15 minutes; person and membership re-checked each request.
Stolen refresh cookie	Single use; reuse after the 10 s grace revokes the whole family.
CSRF on the cookie route	`sameSite: 'lax'` and a single-origin deployment; refresh only returns a token the attacker cannot read.
Password guessing / email enumeration	Rate limit; identical errors and timing.
Mass assignment	zod `.strict()` schemas.
Privilege escalation through members	Only owners assign owner; you cannot edit yourself; a school always keeps an active owner; roles resolved inside the caller's school.
Forgetting to protect a route	Protected by default (order in `app.ts`).

Known gaps: no email verification; anyone can create schools (rate limit only); HTTPS is required in production; no two-factor; the database user has full rights on its own database (fine for now); add Row-Level Security before real

student data; a soft-deleted user keeps their email (a partial unique index fixes it).

D4. Glossary

Tenant / organization	One customer of the system: a school (or a group of schools).
Membership	"This person belongs to this school"; roles hang off it.
Role / permission	A named bundle of capabilities / one capability like `students.create`.
Access token (JWT)	A signed, short-lived token saying who you are and which school you act in.
Refresh token / cookie	A long-lived, single-use secret, kept in an httpOnly cookie, used to get new access tokens.
Token family	The chain of refresh tokens from one sign-in; revoked together.
httpOnly cookie	A cookie JavaScript cannot read; the browser sends it automatically.
Middleware	An Express function that runs before a route and can reject or continue the request.
zod schema	A description of valid input that validates data and yields a TypeScript type.
Migration	A versioned SQL script that changes the database structure (`migrate dev` creates it locally; `migrate deploy` applies it elsewhere).
Transaction	Database changes that all succeed or all roll back.
Trusted sources	DigitalOcean's firewall list of who may connect to the database.

D5. Progress tracker

Print this page or copy it to a note. Tick each line when its **Check** passes.

☐	Step	Est.
☐	C0 DigitalOcean database, user, trusted sources, grants; local Docker database	45 min
☐	C1 project setup, `.env`, git, first commit	15 min
☐	Part A fast track (A1–A4 starred parts; exercises 1, 4, 7)	2.5 h
☐	C2 data model, migration (local)	60 min
☐	C3 foundations; C4 skeleton app	75 min
☐	C5 session engine	60 min
☐	C6 sign up; C7 sign in	75 min
☐	C8 authenticate + /me; C9 refresh + logout	75 min
☐	C10 schools: catalogue, seed, create, list, switch, current	2 h
☐	C11 roles, permissions, members	2.5 h
☐	C12 frontend: F1–F6 (must), F7–F8 (should)	3.5 h
☐	C13 deploy: database release, droplet, nginx, frontend flags	2 h
☐	C14 acceptance checklist; fixes; commit and tag	1 h

About this guide

It contains explanations, specifications and short teaching snippets only — you write the backend. The snippets, curl commands and the DigitalOcean steps are written to be accurate but were **not run against your cluster or your droplet**; the frontend file names and response shapes come from reading your actual code. If a step does not behave as described, send me the exact error and what you did. A tested reference solution of the earlier (simpler) version exists and can be shared if you are stuck — only if you ask for it.